**POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY**
**Faculty of Information Technology**

# Intelligent System Development

## GRAPH 01

### A Diabetes Knowledge Graph from Vinmec Health Articles with Neo4j

**Instructor / Lecturer:** Assoc. Prof. Dinh Que Tran, Ph.D.
**Email:** quetd@ptit.edu.vn, tdque@yahoo.com

**Student Name:** Nguyễn Văn Trường
**Student ID:** B23DCCE095
**Semester:** I.2025 – 2026
**Submission:** Individual Project

**Hanoi, 2026**

# Contents


**1 Introduction** **3**
- 1.1 The Application Problem . . . 3
- 1.2 Why Keyword Search Falls Short . . . 3
- 1.3 Why a Knowledge Graph . . . 3
- 1.4 Objectives and Scope . . . 4
- 1.5 Overview of the Approach . . . 4

**2 Problem Definition** **4**
- 2.1 Input and Output . . . 4
- 2.2 Questions the System Must Answer . . . 5
- 2.3 Success Criteria . . . 5
- 2.4 What the System Is Not . . . 5

**3 Data Collection** **6**
- 3.1 Why Vinmec . . . 6
- 3.2 Finding the Pages . . . 6
- 3.3 Why a Second Filter Is Needed . . . 6
- 3.4 Parsing a Page . . . 7
- 3.5 Data Volume . . . 8

**4 Knowledge Graph Design** **9**
- 4.1 Facts, Entities and Relations . . . 9
- 4.2 Schema: Entity Types and Relation Types . . . 10
- 4.3 Extraction Method . . . 12
  - 4.3.1 Why a bullet point should not become a node . . . 12
  - 4.3.2 The vocabulary and entity normalisation . . . 13
  - 4.3.3 Matching aliases in text . . . 13
  - 4.3.4 The section rule: the heading decides . . . 14
  - 4.3.5 Which diabetes is the subject? . . . 15
  - 4.3.6 Counting evidence: why one page is not enough . . . 16
  - 4.3.7 Foods: recommended or avoided . . . 16
  - 4.3.8 Drug side effects . . . 17
  - 4.3.9 Background facts added by hand . . . 17
- 4.4 Output Files . . . 17
- 4.5 Meeting the Lecture's Requirements . . . 18

**5 System Implementation** **18**
- 5.1 Project Structure . . . 18
- 5.2 Storing the Graph in Neo4j . . . 19
  - 5.2.1 Why a graph database . . . 19
  - 5.2.2 Data model . . . 19
  - 5.2.3 Loading . . . 20
- 5.3 Querying with Cypher . . . 20
- 5.4 Graph Retrieval . . . 20
- 5.5 The Website . . . 21
  - 5.5.1 Architecture . . . 21
  - 5.5.2 API . . . 22
  - 5.5.3 Web client . . . 22

**6 Visualisation** **23**
- 6.1 Size of the Graph . . . 23
- 6.2 The Whole Graph . . . 24
- 6.3 Focused Views . . . 24
- 6.4 Comparing the Diabetes Types . . . 27
- 6.5 Which Entities Matter Most? . . . 28
- 6.6 What Vinmec Writes About Most . . . 28

**7 Experimental Evaluation** **29**
- 7.1 Query Results . . . 29
  - 7.1.1 One-hop: everything about Metformin . . . 29

7.1.2 Two-hop: which organs can type 2 diabetes damage? . . . 30
7.1.3 Comparing type 1 and type 2 symptoms . . . 30
7.1.4 A symptom checker . . . 31
7.1.5 Diagnosis: tests and thresholds . . . 31
7.1.6 Diet for gestational diabetes . . . 32
7.1.7 Provenance: where does a fact come from? . . . 32
7.1.8 Co-occurrence through articles . . . 33
7.2 Correctness of the Facts . . . 33
7.2.1 What was measured and how . . . 33
7.2.2 Result . . . 34
7.3 How Much Evidence the Facts Have . . . 34

**8 Demo** **34**
8.1 Scenario 1: Finding an Entity . . . 35
8.2 Scenario 2: Everything About One Entity, With Sources . . . 37
8.3 Scenario 3: Comparing the Forms of Diabetes . . . 38
8.4 Scenario 4: Checking Symptoms . . . 39
8.5 Scenario 5: How Are Two Things Connected? . . . 40

**9 Error Analysis** **40**
9.1 The Two Errors in the Sample . . . 41
9.2 The Fix . . . 41
9.3 What the Fix Cost: Precision Versus Recall . . . 41
9.4 Other Known Errors . . . 42

**10 Limitations** **42**

**11 Conclusion and Future Work** **43**
11.1 What Was Achieved . . . 43
11.2 Future Work . . . 44

**A Reproduction** **44**

# 1 Introduction

## 1.1 The Application Problem

Diabetes is among the most common chronic diseases in Vietnam, and many people learn about it from health articles online. Vinmec, one of the largest private hospital groups in the country, publishes nearly a thousand Vietnamese articles on diabetes alone: symptoms, who is at risk, complications, diagnosis, drugs, diet.

That knowledge is valuable but locked in text. A reader who asks *which complications of type 2 diabetes affect the eyes?* has to locate the right pages and assemble the answer personally. The same symptom is spelled differently from article to article (*uống nhiều*, *khát nước*, *hay khát*), and an answer is often split: one article says type 2 can cause retinopathy, another that retinopathy damages the retina. For a program the situation is worse, because an article is merely a long string of characters.

The goal is to convert the article collection into a form that programs can query and people can explore without reading hundreds of pages, while keeping the link to the original articles so that any answer can be checked.

## 1.2 Why Keyword Search Falls Short

Keyword search is the obvious tool for a document collection, and it fails here in three ways.

**Many spellings for one thing.** Searching *khát nước* (thirsty) misses an article that writes *uống nhiều* or *khát nhiều*; searching *Metformin* misses *Glucophage*. The reader would need to know every synonym.

**A word is not a fact.** A breakfast article may say that eating well "helps you feel less *mệt mỏi*" (tired). A search for *mệt mỏi* returns it, yet the article does not claim fatigue is a symptom of diabetes. Search locates words, not statements.

**Two-fact answers are out of reach.** No article states "type 2 diabetes damages the kidney" in those words; the answer joins "type 2 can cause diabetic kidney disease" with "diabetic kidney disease affects the kidney". Search returns documents one at a time and cannot join statements from different ones.

## 1.3 Why a Knowledge Graph

A Knowledge Graph stores knowledge as many small **facts**, each linking two things by a named relation: *(subject, relation, object)*. Drawn out, the things are dots (**entities**, or nodes) and the facts are arrows (**relationships**, or edges), so the collection forms a network.

This answers the three failures directly:

- **Synonyms are resolved once.** While facts are created, *uống nhiều*, *khát nước* and *hay khát* are all mapped to the single entity *Khát nhiều*, and every later question uses it.
- **Facts are explicit and typed.** A fact exists only if an article states it where such a statement is made (Section 4.3).
- **Facts combine by following arrows.** A question becomes a walk: start at *type 2*, follow *can cause* to the complications, then *affects* to the organs. The path is the explanation.

The lecture contrasts the two retrieval styles: vector or keyword retrieval asks "which passages resemble the question?", graph retrieval asks "which entities and relationships are explicitly connected?". For questions about how symptoms, causes, complications, tests and treatments relate, the second is the right question.

A graph also keeps what a trained model loses, **provenance**. Each fact here records the articles that state it, so an answer can be verified, which matters for a medical topic where a confident wrong answer can mislead a patient.

### 1.4 Objectives and Scope

The task is to build a diabetes Knowledge Graph from Vinmec data, store and query it with Neo4j, and provide a website to retrieve it. Table 1 lists the requirements, taken from the task and from the lecture's Mini Project 3 and Exercise 1, with the section that addresses each.

Table 1: Requirements and where they are addressed.

| # | Requirement | Section |
|---|---|---|
| R1 | Collect data about diabetes from Vinmec | 3 |
| R2 | Build a Knowledge Graph: $\geq$ 50 entities, $\geq$ 100 relationships, $\geq$ 5 relationship types, $\geq$ 3 entity types | 4, 4.5 |
| R3 | Store the triples as CSV in the form `subject, relation, object` | 4.4 |
| R4 | Store and query the graph in Neo4j; at least 5 Cypher queries, including one-hop and two-hop | 5.2, 7.1 |
| R5 | Visualise the graph | 6 |
| R6 | Graph retrieval from an entity within 1–2 hops | 5.4 |
| R7 | A website for retrieving the data | 5.5, 8 |
| R8 | Evaluate the system systematically | 7 |
| R9 | Analyse errors and limitations | 9, 10 |

The report follows the lecture's *Recommended Final Report Structure* (§37 of *From Deep Learning to Knowledge Graph, RAG and Intelligent Agents*). That outline covers a full intelligent assistant, with sections on a deep-learning model, a RAG architecture and an agent architecture. This project is the Knowledge Graph and retrieval-website part, so those three sections are omitted. Two are added because they hold a large share of the work: **Visualisation** (Section 6) and **Demo** (Section 8).

### 1.5 Overview of the Approach

Figure 1 shows the system. Every box is a separate program that reads the previous one's output from disk. The split is deliberate. Crawling takes about twenty minutes and needs the site to be reachable, so it runs once. Extraction is where the rules were revised and retested several times, so it must rerun cheaply without downloading again. Loading can rebuild the database from the CSV files at any moment, and the website only reads from the database.

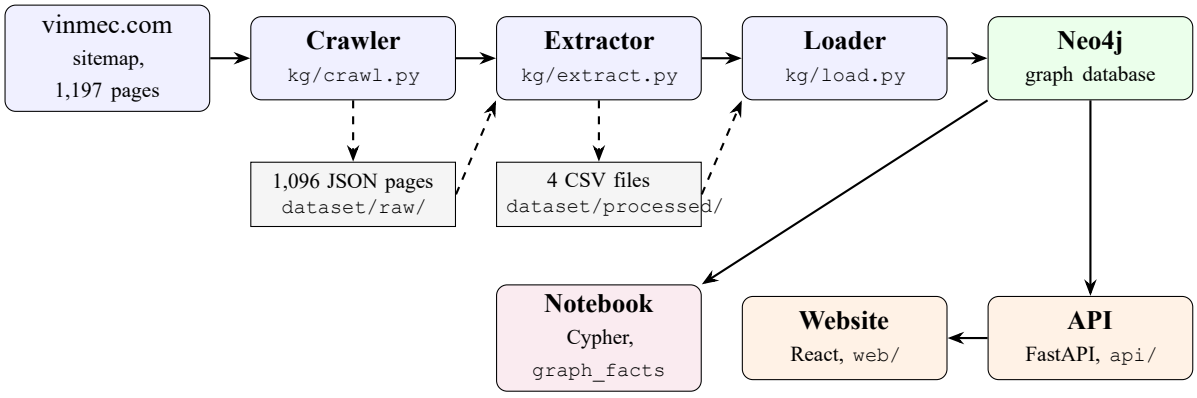



Figure 1: System overview. Solid arrows show data flowing between programs, dashed arrows files written to and read from disk. The notebook, the Cypher queries and the website all read the same Neo4j database.

## 2 Problem Definition

### 2.1 Input and Output

**Input.** The public Vietnamese diabetes articles of vinmec.com: the official disease pages (short structured summaries with fixed headings for overview, causes, symptoms, risk groups, prevention, diagnosis and

treatment) and the much larger set of advice articles written or reviewed by Vinmec doctors. No other source is used.

**Output.** A Knowledge Graph in which

- each node is a named diabetes concept with a type: a form of diabetes, symptom, risk factor, complication, test, drug, drug class, food, lifestyle measure or organ;
- each edge is a fact with a named relation such as `HAS_SYMPTOM` or `TREATED_BY`;
- each extracted fact records how many different articles state it (its **support**) and which ones;

together with three ways to use it: Cypher queries in Neo4j, a retrieval function returning the facts around one entity, and a website.

## 2.2 Questions the System Must Answer

A graph is useful only if it answers real questions. The design started from what a reader of diabetes articles would ask, and each question calls for a different query shape (Table 2).

Table 2: Questions the system must answer and the query each one needs.

| Question | Query type | What the graph must hold |
|---|---|---|
| What are the symptoms of type 1 diabetes? Which drugs treat type 2? | one-hop | typed facts from each diabetes type |
| What do we know about Metformin? | one-hop, both directions | facts where Metformin is subject or object |
| Which organs can type 2 diabetes damage? | two-hop | complications *and* the organ each affects |
| Which types of diabetes match these symptoms? | aggregation over one hop | symptom facts per type, with support |
| How is obesity connected to the kidney? | shortest path | a connected graph |
| How is diabetes diagnosed, at which values? | one-hop + node property | tests with thresholds on the test nodes |
| Where does this fact come from? | provenance | article URLs stored on each fact |

These questions shaped the design. The two-hop question is why *affects organ* facts were added by hand (Section 4.3.9); the symptom checker is why support is kept on every fact instead of being used once as a filter; the provenance question is why articles are nodes in the graph.

## 2.3 Success Criteria

1. **Size and structure** (Mini Project 3, Exercise 1): at least 50 entities, 100 relationships, 5 relation types, 3 entity types; triples in CSV; at least five Cypher queries including one-hop and two-hop; a visualisation.
2. **Correctness**, measured and not assumed: a random sample of facts is checked by hand against the source articles, and precision is reported with its uncertainty.
3. **Traceability**: every extracted fact links to the articles behind it.
4. **Usability**: someone who knows no Cypher can find an entity, see its facts and sources, compare diabetes types, check symptoms and find connections.

## 2.4 What the System Is Not

The graph summarises what public health articles say. It is neither a diagnostic tool nor medical advice. “Correct” in this report means that the Vinmec articles state the fact for that form of diabetes, not that a

physician verified it against current guidelines. The website and the evaluation both repeat this, since on a medical topic the difference matters.

# 3 Data Collection

## 3.1 Why Vinmec

Three properties make Vinmec a good single source for a Vietnamese diabetes graph. The articles are in Vietnamese, so the graph uses the words readers actually use. Coverage is wide: nearly a thousand diabetes articles span early signs, diet, drugs and pregnancy. And the writing is consistent: doctors write or review it, and it follows recognisable headings such as *Triệu chứng* (symptoms), *Nguyên nhân* (causes) and *Điều trị* (treatment). Section 4.3 shows that this regularity is what makes automatic extraction feasible.

## 3.2 Finding the Pages

**Why not the search box.** A program cannot use the site's search: results are drawn by JavaScript in the browser, so downloading the search page yields the page before any results exist. A test request for "tiểu đường" returned about 150 KB of HTML with no article links.

**The sitemap.** Websites publish a **sitemap**, a file listing every URL so that search engines can index them. Vinmec's `sitemap.xml` points to about 230 smaller sitemaps. The crawler reads those for Vietnamese articles (`posts-vi-*`) and Vietnamese disease pages (`diseases-vi-*`), which together list around 222,000 URLs: the whole Vietnamese site.

**Selecting by slug.** Each URL ends in a readable name called the **slug**, written without accents; the article *"Bệnh đái tháo đường: chế độ ăn như thế nào?"* has the slug `benh-dai-thao-duong-che-do-nhu-nao-vi`. Selecting by slug avoids downloading all 222,000 pages. Keywords come in two strengths because their risks differ:

- **strong** (`tieu-duong`, `thao-duong`, `dai-duong`): a slug containing one is almost certainly about diabetes;
- **weak** (`insulin`, `metformin`, `hba1c`, `duong-huyet` = blood sugar, `glucose`, `sglt2`…): the page *may* be about diabetes, but these words also occur in articles on other subjects, such as the role of insulin in the body or blood sugar in healthy people.

The slug `thao-nhat` is excluded: *đái tháo nhạt* (diabetes insipidus) shares "đái tháo" with *đái tháo đường* but is a different disease of kidneys and hormones.

This yields **1,197 candidate URLs**. A few either failed to download or duplicated an article under a second address (the crawler skips a title it has already seen), so **1,191 pages** were downloaded and checked: 944 with a strong keyword and 247 with only a weak one.

**Politeness.** The site's `robots.txt`, which states what automated programs may fetch, permits these paths. The crawler identifies itself as a student research crawler, waits one second between requests, retries failures with growing pauses, saves each page as soon as it is processed and skips pages already saved, so an interrupted crawl resumes where it stopped.

## 3.3 Why a Second Filter Is Needed

Selecting by slug has a flaw caused by the missing accents. Vietnamese has many words that differ only by accent, and without accents they coincide:

- `tieu-duong` is *tiểu đường* (diabetes), but also the start of *tiểu dương vật* (a urology question about a child);

- `dai-duong` is *đái đường*, but also *đại dương* (ocean), as in *cá ngừ đại dương*;
- `duong` also hides *dưỡng ẩm* (moisturising) and *dương tính* (a positive test result).

These pages pass the slug test and have nothing to do with diabetes. After downloading, each page is therefore rechecked on its real text, **with accents**, where such words are distinct. The check counts how often the page mentions diabetes (*tiểu đường*, *đái tháo đường*, *ĐTĐ*: the "diabetes count") and how often it uses related terms (*đường huyết*, *insulin*, *HbA1c*, *biến chứng*…: the "related count"). A page is kept if:

1. it is an official Vinmec **disease page**, already classified by Vinmec; or
2. its **title or summary** mentions diabetes and the text does so at least twice (the title is the author's own statement of the topic, the strongest signal); or
3. its slug had a **strong** keyword and the text mentions diabetes at least three times, which removes accent collisions (they mention it zero times); or
4. its slug had only a **weak** keyword and the text mentions diabetes at least five times *and* uses related terms at least five times. An article "Insulin là gì?" that names the hormone thirty times and diabetes once is about biology, not the disease.

The thresholds are loose where evidence is strong (the title) and strict where a wrong page is likely (a weak keyword). Table 3 gives the outcome and Table 4 some rejected pages.

Table 3: Candidate pages kept and rejected by the relevance filter, by slug keyword strength.

| Slug keyword | Kept | Rejected | Checked |
|---|---|---|---|
| strong | 934 | 10 | 944 |
| weak | 162 | 85 | 247 |
| **All** | 1096 | 95 | 1191 |

Table 4: Examples of rejected pages with their diabetes and related counts. The first four are accent collisions.

| Title | Slug keyword | Diabetes count | Related count |
|---|---|---|---|
| Trẻ 19 tháng đi tiểu dương vật không cử động cần làm gì? | strong | 0 | 0 |
| Vô tình hít phải Heroin nên đi thử nước tiểu dương tính có phải không? | strong | 0 | 0 |
| Thành phần dinh dưỡng trong cá ngừ đại dương | strong | 2 | 0 |
| Dầu hoa anh thảo dưỡng ẩm da có tốt không? | strong | 0 | 0 |
| Cơm rượu nếp cẩm có làm tăng chỉ số đường huyết không? | weak | 4 | 9 |
| Chỉ số Cholesterol: 222, Triglyceride: 74, HDL: 75, LDL: 134 và Glucose: 105 có cần uống thuốc không? | weak | 4 | 7 |
| Điều trị tăng Triglyceride bằng insulin | weak | 4 | 14 |

Only 10 of 944 strong-keyword pages were rejected, all of them collisions like *tiểu dương vật* and *đại dương*. Of the weak-keyword pages, about a third were rejected as being about insulin, blood sugar or tests in general. The filter is imperfect: a short article on a winter-melon remedy for diabetes falls under the threshold. Losing a few short pages costs little, while admitting off-topic ones would put false facts into the graph.

### 3.4 Parsing a Page

The crawler does not store raw HTML, in which menus, advertisements, booking forms, footers and scripts outweigh the article. It uses the structure of Vinmec's pages:

- **articles**: the body is the element of class `entry`; title, date, author and summary come from the structured metadata (`application/ld+json`) that search engines read;
- **disease pages**: each section is an element of class `item_detial_sick` holding a heading and a body.

The body is split into **sections**, and for each the crawler saves the **heading**, the **paragraphs** and the **bullet points** separately.

Keeping the heading is the most consequential design decision in the project. The heading says what kind of information the text below it holds: the bullet "Mệt mỏi" (fatigue) means one thing under *Triệu chứng* and another under *Tác dụng phụ của thuốc* (side effects). Section 4.3 builds on this.

Text at the end of an article that is not part of it is removed: the booking message (*"Để đặt lịch khám tại viện, Quý khách vui lòng bấm số HOTLINE…"*), the app advert and the "Xem thêm:" (see also) links. Left in, every article would "mention" the same booking phrases and the links would import words from unrelated topics. Figure 2 shows the saved form of the type 2 disease page.

```
title: Tiểu đường tuýp 2: Nguyên nhân, triệu chứng, chẩn đoán và điều trị
url:   https://www.vinmec.com/vie/benh/tieu-duong-tuyp-2-3042     type: disease

## Tổng quan bệnh Tiểu đường tuýp 2            (4 paragraphs, 0 bullets)
## Nguyên nhân bệnh Tiểu đường tuýp 2          (1 paragraph,  0 bullets)
## Triệu chứng bệnh Tiểu đường tuýp 2          (3 paragraphs, 4 bullets)
     • Ăn nhiều
     • Uống nhiều (hay khát nước)
     • Tiểu nhiều (do đường trong nước tiểu cao, gây lợi niệu thẩm thấu)
     • Gầy nhiều (gầy sút cân): bệnh nhân tiểu đường tuýp 2 thường có ...
## Biến chứng cấp tính                         (1 paragraph,  0 bullets)
## Biến chứng mạn tính                         (0 paragraphs, 6 bullets)
     • Nhìn mờ (do biến chứng võng mạc, đục thủy tinh thể)
     • Tê bì dị cảm ở bàn chân (biến chứng thần kinh)
     ...
## Đối tượng nguy cơ bệnh Tiểu đường tuýp 2    ...
## Các biện pháp chẩn đoán bệnh Tiểu đường tuýp 2  ...
## Các biện pháp điều trị bệnh Tiểu đường tuýp 2   ...
```

Figure 2: Vinmec's type 2 disease page as saved: sections, each with heading, paragraphs and bullets (abridged).

### 3.5 Data Volume

Table 5: Size of the collected corpus.

| Quantity | Value |
|---|---|
| Candidate URLs from the sitemap | 1,197 |
| Pages downloaded and checked | 1,191 |
| Pages kept (about diabetes) | 1,096 |
| Official disease pages among them | 4 |
| Total words in kept pages | 1,130,914 |
| Median words per page | 1044 |
| Median diabetes mentions per page | 16 |
| Pages with a publication date / dated 2024 | 1,092 / 1,083 |

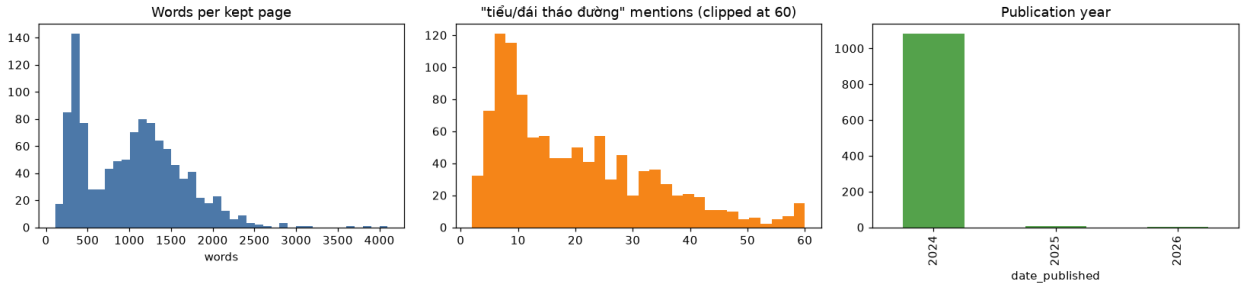



Figure 3: Left: words per kept page. Middle: diabetes mentions per page (clipped at 60). Right: publication year of kept pages.

Figure 3 sketches the corpus. A typical page runs to about a thousand words, enough for several aspects of the disease under separate headings. Half of the kept pages mention diabetes 16 times or more and few mention it fewer than six times, which confirms the filter kept pages about the disease and not passing mentions. Nearly all dated pages (1,083 of 1,092) carry a 2024 date. That is when Vinmec republished its archive, not when each article was written, so true article age is unknown; Section 10 returns to this.

# 4 Knowledge Graph Design

## 4.1 Facts, Entities and Relations

A Knowledge Graph is a set of **triples**, each stating one fact in a fixed three-part form: what the fact is about (the **subject**), the kind of link (the **relation**) and the other thing (the **object**). The part of the type 2 disease page that lists classic symptoms and chronic complications becomes:

| subject | relation | object |
|---|---|---|
| Đái tháo đường type 2 | HAS_SYMPTOM | Khát nhiều |
| Đái tháo đường type 2 | HAS_SYMPTOM | Tiểu nhiều |
| Đái tháo đường type 2 | HAS_COMPLICATION | Bệnh võng mạc |
| Bệnh võng mạc | AFFECTS_ORGAN | Mắt |

This beats the paragraph because every fact has the same predictable form. A program need not parse Vietnamese grammar; it looks for facts whose subject is "type 2" and relation `HAS_COMPLICATION`. The ambiguity of language is settled once, when the fact is made, not at every question.

Draw each thing as a dot and each fact as an arrow and the facts become a **graph**: the things are **nodes** or **entities**, the facts **edges** or **relationships**, and each node has a **type** (`Symptom`, `Drug`…). Figure 4 draws seven hand-written facts. Questions become walks: "which organ does type 2 damage?" goes from *type 2* along `HAS_COMPLICATION` to *Bệnh thận đái tháo đường*, then along `AFFECTS_ORGAN` to *Thận*. No single fact holds the answer; two facts joined at the middle node do. The lecture calls this a **two-hop** query, as opposed to a **one-hop** query that reads only the arrows touching one node.

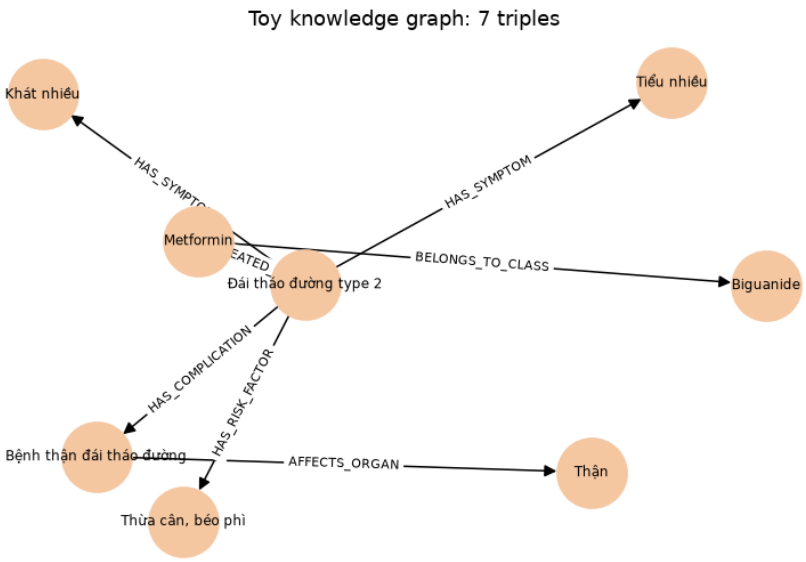



Figure 4: Seven hand-written diabetes facts as a graph (notebook, Part 1). The kidney (*Thận*) is two hops from *type 2*: no single fact joins them.

The real graph has the same shape but is built automatically from 1,096 articles. The hard part of the project is not storing the graph but extracting **correct** facts from free Vietnamese text without anyone reading every article, and the rest of this section explains how.

## 4.2 Schema: Entity Types and Relation Types

The schema says which kinds of entities exist and which facts may link them. It was designed from the questions of Table 2 and from the layout of Vinmec's articles.

**Entity types.** There are ten (Table 6). Five mirror the standard sections of a disease article: symptoms, risk factors, complications, tests and treatments. Treatments split into `Drug` and `Lifestyle` since the questions differ ("which drug class?" makes sense only for drugs). `Food` is separate because diet is among the most discussed topics. `DrugClass` and `Organ` are not extracted from text: they group drugs and place complications in the body (Section 4.3.9). The five `DiabetesType` nodes are the subjects of almost every extracted fact.

Table 6: Entity types: vocabulary entries, aliases (spellings), entities that reached the graph, and the best-connected examples.

| Type | Vocabulary | Aliases | In graph | Examples |
|---|---|---|---|---|
| Drug | 22 | 42 | 22 | Insulin, Metformin, Insulin glargine |
| Complication | 19 | 52 | 18 | Bệnh tim mạch, Bệnh thận đái tháo đường, Bệnh võng mạc |
| Food | 16 | 66 | 16 | Các loại hạt, Cơm trắng, tinh bột tinh chế, Ngũ cốc nguyên hạt |
| RiskFactor | 17 | 52 | 16 | Di truyền / tiền sử gia đình, Hội chứng buồng trứng đa nang, Kháng insulin |
| Symptom | 16 | 56 | 16 | Mệt mỏi, Tiểu nhiều, Khát nhiều |
| Test | 10 | 35 | 10 | HbA1c, Nghiệm pháp dung nạp glucose (OGTT), Đường huyết lúc đói |
| DrugClass | – | – | 9 | Nhóm Insulin, Ức chế DPP-4, Sulfonylurea |
| Organ | – | – | 9 | Mắt, Thần kinh, Bàn chân |
| Lifestyle | 6 | 17 | 6 | Giảm cân, Khám sức khỏe định kỳ, Tập thể dục đều đặn |
| DiabetesType | 5 | 19 | 5 | Đái tháo đường, Đái tháo đường type 2, Đái tháo đường thai kỳ |

Nearly every vocabulary entry earned enough support to enter the graph; the one complication and one risk

factor that did not are discussed in Section 9.

**Relation types.** Table 7 lists the twelve relations with the entity types they join and their fact counts. The `source` column separates facts extracted from articles (`vinmec`) from general classifications added by hand (`curated`).

Table 7: Relation types in the graph.

| Relation | From → to | Source | Facts |
|---|---|---|---|
| HAS_COMPLICATION | DiabetesType → Complication | vinmec | 52 |
| TREATED_BY | DiabetesType → Drug | vinmec | 46 |
| HAS_SYMPTOM | DiabetesType → Symptom | vinmec | 45 |
| HAS_RISK_FACTOR | DiabetesType → RiskFactor | vinmec | 42 |
| RECOMMENDS_FOOD | DiabetesType → Food | vinmec | 35 |
| DIAGNOSED_BY | DiabetesType → Test | vinmec | 30 |
| AVOIDS_FOOD | DiabetesType → Food | vinmec | 26 |
| BELONGS_TO_CLASS | Drug → DrugClass | curated | 22 |
| MANAGED_BY | DiabetesType → Lifestyle | vinmec | 19 |
| AFFECTS_ORGAN | Complication → Organ | curated | 12 |
| HAS_SIDE_EFFECT | Drug → Complication/Symptom | vinmec | 6 |
| SUBTYPE_OF | DiabetesType → DiabetesType | curated | 4 |

The direction of each relation is a modelling choice: `HAS_SYMPTOM` runs from disease to symptom but could be `SYMPTOM_OF` the other way. The convention here is that facts point *away* from the diabetes types and on to classifications. Section 6.5 shows this choice matters for graph measures. Figure 5 draws the schema: each arrow is a question the graph can answer, and a missing arrow (say `Food` to `Complication`) is a question it cannot.

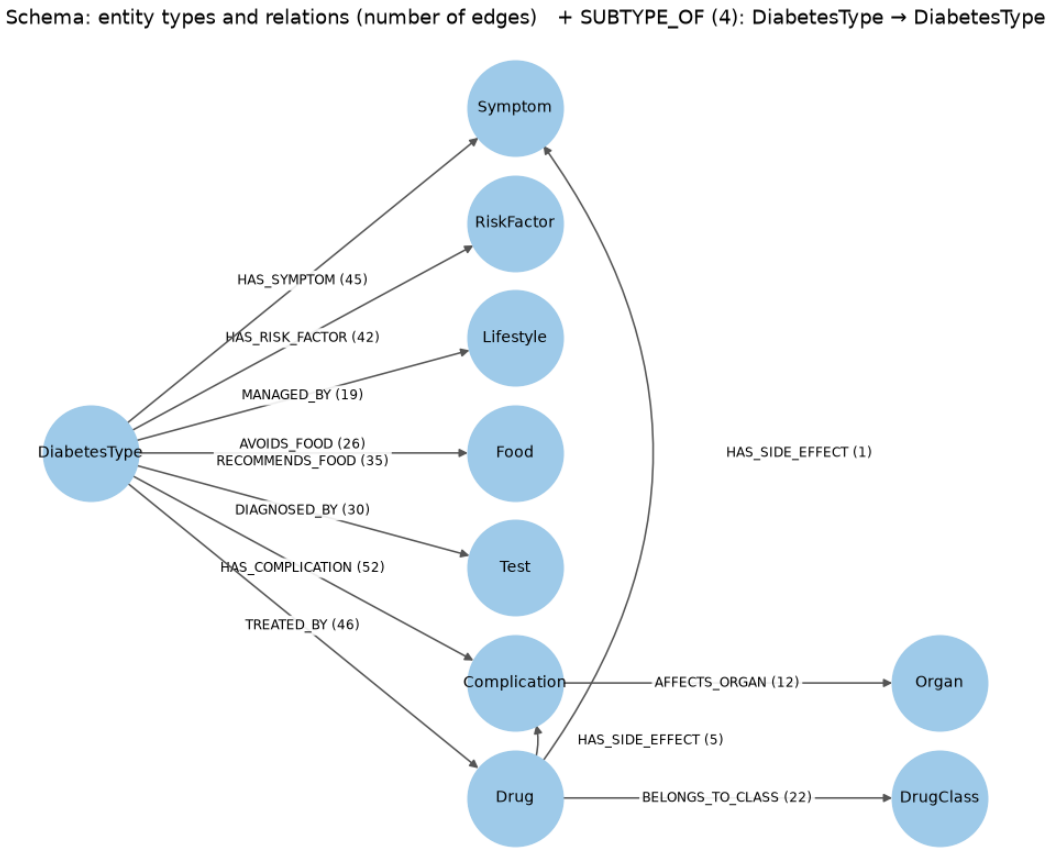



Figure 5: The schema as stored in Neo4j: entity types (circles) and the relations between them with fact counts. Diabetes types on the left, what the articles say about them in the middle, hand-added classifications on the right.

## 4.3 Extraction Method

### 4.3.1 Why a bullet point should not become a node

The simplest scheme would make every bullet under *Triệu chứng* a Symptom node. On the type 2 page that gives acceptable nodes such as "Ăn nhiều". But a bullet under *Điều trị* on the type 1 page reads:

> *"Kiểm soát đường huyết: đối với tuýp 1 thì kiểm soát bằng insulin ngoại sinh là chủ yếu. Dùng theo phác đồ, tuân thủ giờ tiêm và liều lượng tránh bị tụt đường huyết. Các loại insulin gồm có…"*

Sixty words make a useless node: no other article repeats them, so it links to nothing and matches no question. Even short bullets differ between articles (*uống nhiều*, *khát nước*, *hay khát*), so a single fact "diabetes causes thirst" would shatter into weak pieces that never meet.

The project therefore decides **in advance** which concepts the graph may contain, names each cleanly, and searches the text for every way of writing it. For every section of every page, extraction then answers three questions:

1. **Which diabetes is the page about?** That is the subject.
2. **What may this section contain?** The heading decides.
3. **Which known concepts does it mention?** Those are the objects.

Each fact found on a page is one **vote** from that page; votes are tallied, and a fact enters the graph only when enough different pages agree. Figure 6 traces a real section.

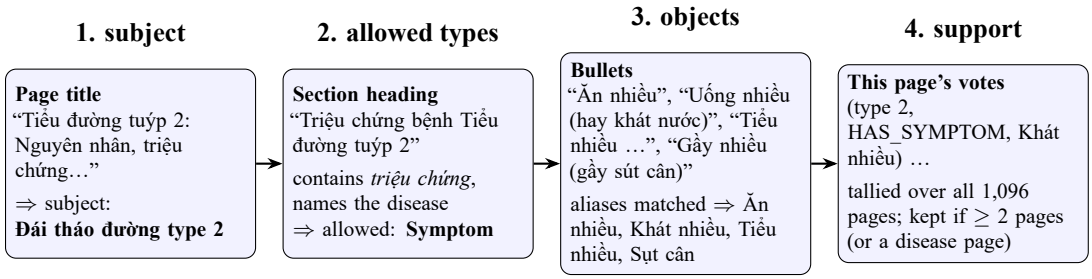



Figure 6: Extracting symptom facts from the symptom section of Vinmec’s type 2 disease page.

### 4.3.2 The vocabulary and entity normalisation

The vocabulary (`kg/vocab.py`) lists 106 concepts, each with a clean **canonical name** that becomes the node, a **type**, and **aliases**, the spellings found in articles (320 in all). A sample:

```
VOCAB = {
    "Khát nhiều":     ("Symptom", ["khát nhiều", "uống nhiều", "khát nước", "hay
    ↪  khát", "khô miệng"]),
    "Tê bì tay chân": ("Symptom", ["tê bì", "tê tay", "tê chân", "dị cảm", "ngứa
    ↪  ran", "kiến bò"]),
    "Kháng insulin": ("RiskFactor", ["kháng insulin", "đề kháng insulin"]),
    "Bệnh võng mạc": ("Complication", ["võng mạc", "biến chứng mắt", "mù lòa"]),
    "HbA1c":          ("Test", ["hba1c", "hemoglobin a1c", "a1c"]),
    "Metformin":      ("Drug", ["metformin", "glucophage"]),
    "Empagliflozin": ("Drug", ["empagliflozin", "jardiance"]),
    "Rau xanh":       ("Food", ["rau xanh", "rau lá xanh", "rau củ", "bông cải",
    ↪  "súp lơ"]),
    ...
}
```

Mapping many spellings to one node is **entity normalisation**, and it is what lets evidence accumulate: an article saying *uống nhiều* and another saying *khát nước* both vote for *(diabetes, HAS_SYMPTOM, Khát nhiều)*. Brand names (*Glucophage*, *Diamicron*, *Januvia*, *Jardiance*, *Lantus*) are aliases because Vietnamese articles often name drugs by brand.

The list was written by hand from the topics the articles cover: classic symptoms, risk factors on Vinmec’s pages, acute and chronic complications, standard tests, drugs used in Vietnam, and the foods and lifestyle measures of diet articles. The price is plain, a concept missing from the list can never be found, and so is the benefit, every node is clean, meaningful and correctly typed. Section 10 weighs the price.

### 4.3.3 Matching aliases in text

All aliases of an entity are compiled into one regular expression. For *Tiểu nhiều*:

```
(?<!\w)(đi tiểu thường xuyên|tiểu thường xuyên|đi tiểu nhiều|tiểu nhiều|tiểu
↪  đêm)(?!\w)
```

Three details matter.

**Whole words only.** Vietnamese separates syllables with spaces, and many aliases are one short syllable that also occurs inside longer words: the fruit *lê* (pear) would match inside *lên* (as in *lên cân*, gaining weight) and *bia* (beer) inside *biên*. The lookarounds `(?<!\w)` and `(?!\w)` demand non-letters on both sides, so “bệnh nhân lên cân” contains no food while “ăn quả lê” does.

**Longest alias first.** Were “tiểu thường xuyên” tried before “đi tiểu thường xuyên”, the short form would win and the more specific wording never be used.

**Accents stay.** Text is lower-cased so "HbA1C" equals "hba1c", but accents are never stripped, which would revive the URL problem (*tiểu đường* equals *tiểu dương*).

Word boundaries cannot stop an alias that is a whole word with a second meaning: the fruit *táo* (apple) matches *táo bón* (constipation). The section rule below keeps such errors rare, because a food alias counts only inside diet sections.

#### 4.3.4 The section rule: the heading decides

Finding a vocabulary word in an article does not mean the article asserts something about it. Real cases from the corpus:

- *mệt mỏi* appears in a diet article ("ăn sáng đầy đủ giúp bạn đỡ mệt mỏi"), which does not say fatigue is a symptom of diabetes;
- *tăng huyết áp* appears in drug paragraphs as a condition the drug should not be combined with;
- nearly every article says *insulin* while explaining blood sugar, not because insulin is the treatment under discussion.

If every match counted, diabetes would acquire dozens of "symptoms" that are just words that happened to occur. A match therefore counts only when the **section heading** says the section is about that kind of information, which works because hospital articles are organised under exactly such headings (Table 8).

Table 8: The section rule: heading keywords and the entity types extractable below them.

| Heading contains | Allowed types | Relation produced |
|---|---|---|
| triệu chứng, dấu hiệu, biểu hiện, nhận biết | Symptom | HAS_SYMPTOM |
| nguyên nhân, nguy cơ, đối tượng, yếu tố, ai dễ | RiskFactor | HAS_RISK_FACTOR |
| biến chứng, nguy hiểm, hậu quả, ảnh hưởng | Complication | HAS_COMPLICATION |
| chẩn đoán, xét nghiệm, phát hiện, chỉ số, kiểm tra, theo dõi | Test | DIAGNOSED_BY |
| điều trị, thuốc, insulin, kiểm soát | Drug, Lifestyle | TREATED_BY, MANAGED_BY |
| phòng ngừa, phòng tránh, ngăn ngừa, lối sống, lưu ý | Lifestyle | MANAGED_BY |
| ăn, thực phẩm, dinh dưỡng, thực đơn, món, uống, kiêng, tránh | Food | RECOMMENDS_FOOD / AVOIDS_FOOD |

A section whose heading has none of these keywords, such as *Tổng quan* (overview), yields no facts. This is intended: an overview mentions everything in passing, and missing a fact is preferable to inventing one.

**A second check for symptoms and risk factors.** The quality audit in Section 7.2 exposed a gap in the first version. A heading may contain *nguyên nhân* or *nguy cơ* without being about diabetes:

- *"Một số nguyên nhân gây ngứa bàn tay và bàn chân"* lists causes of **itching**;
- *"Yếu tố nguy cơ gây tai biến mạch máu não ở người tiểu đường"* lists risks of **stroke** in people who already have diabetes;
- *"Làm gì để giảm thiểu nguy cơ tim mạch…"* gives advice on **lowering** a risk.

Reading these bullets as risk factors *of diabetes* produced false facts. A symptom or risk factor always belongs to *something*, and the heading names it. For these two types only, the heading must therefore also pass `about_disease()`:

```
DISEASE_TERMS = ["tiểu đường", "đái tháo đường", "đtđ", "tuýp", "type", "típ",
↪  "bệnh"]
```

```
OTHER_TOPIC   = ["biến chứng", "thận", "võng mạc", "mắt", "thần kinh", "bàn
↪  chân", "tim mạch",
                 "ngứa", "gout", "da", "răng", "nhiễm toan", "ceton", "hạ đường
                 ↪  huyết",
                 "thai nhi", "trẻ sinh ra", "tai biến", "đột quỵ", "mạch vành",
                 ↪  "hôn mê"]
REDUCE_VERBS  = ["giảm", "phòng", "tránh", "ngừa", "hạn chế", "kiểm soát"]

def about_disease(heading):
    """Generic heading ("Triệu chứng", "Ai dễ bị?") or one that names diabetes,
    ↪  and no other topic."""
    h = re.sub(r"^[\d\.\s]+", "", heading.lower())          # drop numbering
    ↪  such as "2.1."
    if any(t in h for t in OTHER_TOPIC) or any(v + " nguy cơ" in h for v in
    ↪  REDUCE_VERBS):
        return False
    return any(t in h for t in DISEASE_TERMS) or len(h.split()) <= 5

def allowed_labels(heading):
    h = heading.lower()
    labels = set()
    for keys, labs in SECTION_RULES:                          # the rule of
    ↪  Table 7
        if any(k in h for k in keys):
            labels |= labs
    if not about_disease(heading):
        labels -= {"Symptom", "RiskFactor"}
    return labels
```

So the heading must name diabetes or be short and generic (“Triệu chứng”, “Ai dễ bị?”, which can only refer to the disease), must name no other topic, and must not be about reducing a risk. Table 9 applies the rule to real headings.

Table 9: The section rule applied to real headings.

| Heading | About diabetes itself? | Allowed types |
|---|---|---|
| Triệu chứng bệnh Tiểu đường tuýp 2 | yes | Symptom |
| Đối tượng nguy cơ bệnh Tiểu đường tuýp 1 | yes | RiskFactor |
| 3. Ai dễ bị tiền tiểu đường? | yes | RiskFactor |
| Các biện pháp điều trị bệnh Tiểu đường tuýp 1 | yes | Drug, Lifestyle |
| 2.2. Một số thực phẩm mẹ bầu mắc bệnh tiểu đường thai kỳ cần tránh | yes | Food |
| 3. Một số nguyên nhân gây ngứa bàn tay và bàn chân | no | – |
| 2. Yếu tố nguy cơ gây tai biến mạch máu não ở người tiểu đường | no | – |
| 4. Làm gì để giảm thiểu nguy cơ tim mạch ở bệnh nhân tiểu đường? | no | – |
| Biến chứng mạn tính | yes | Complication |

### 4.3.5 Which diabetes is the subject?

Diabetes is several diseases. Type 1 is autoimmune loss of insulin-producing cells, usually beginning young. Type 2 develops slowly in adults with weight and lifestyle. Gestational diabetes arises in pregnancy. Prediabetes precedes type 2. Their symptoms, causes and treatments overlap but differ: “autoimmune destruction of beta cells” is a type 1 cause only, “pre-eclampsia” a gestational complication only. One shared “diabetes” node would erase these differences.

Each page therefore gets its subject from the **title**, the author’s own short statement of the topic and more reliable than counting words in the body (a type 2 article often mentions type 1 for comparison). The title is tested in turn for *tuýp 1 / type 1 / típ 1*, *tuýp 2 / type 2*, *thai kỳ / mang thai / mẹ bầu* and *tiền đái tháo đường / tiền tiểu đường*; if none is found, the subject is diabetes in general.

This simplification has a visible effect (Figure 7): most titles name no type, so the general node collects most facts even though many of those articles concern type 2. It is accepted on purpose, because a type 2 fact on the general node is merely less specific, whereas on the type 1 node it would be false.

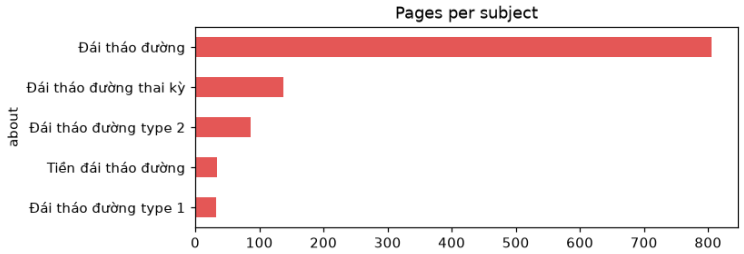



Figure 7: Pages assigned to each subject. Most titles name no specific type.

#### 4.3.6 Counting evidence: why one page is not enough

Every rule above can still fail on one page: an odd heading, a passing mention, a list written for comparison. Trusting each page would turn each slip into a fact. The defence is **agreement between independent pages**. For every fact the extractor keeps the *set* of pages that voted for it (a set, so a page repeating "khát nhiều" five times counts once), and the size of that set is the fact's **support**.

Example, with three pages about type 2:

| Page | Facts it votes for |
|---|---|
| A | type 2 HAS_SYMPTOM Khát nhiều; type 2 HAS_SYMPTOM Mệt mỏi |
| B | type 2 HAS_SYMPTOM Khát nhiều |
| C | type 2 HAS_SYMPTOM Khát nhiều; type 2 HAS_SYMPTOM Run tay |

*Khát nhiều* has support 3 and is kept; *Mệt mỏi* and *Run tay* have support 1 and are dropped, the same logic as checking a claim against a second source. There are two exceptions, each with a reason:

- a fact from an official Vinmec **disease page** is accepted at support 1, since these are short, hospital-reviewed summaries with exactly the standard headings and so the most reliable input;
- a **side effect** needs support 3, because it is read from single sentences and is noisier (below).

```
for (h, rel, t), urls in support.items():                # urls = set of pages
↪  voting for the fact
    need = MIN_SUPPORT_SIDE_EFFECT if rel == "HAS_SIDE_EFFECT" else MIN_SUPPORT
    ↪  # 3 or 2
    if len(urls) >= need or (h, rel, t) in from_disease:
        triples.append((h, rel, t, len(urls), "vinmec", "
        ↪  ".join(sorted(urls)[:5])))
```

The support number survives the filter and is stored on the fact in Neo4j, so queries can rank by documentation, and the website lets the reader hide facts below a chosen level.

#### 4.3.7 Foods: recommended or avoided

The graph distinguishes `RECOMMENDS_FOOD` from `AVOIDS_FOOD`. The first attempt decided from cue words in the sentence or heading (*nên ăn* for recommended; *kiêng*, *tránh*, *hạn chế*, *không nên* for avoid) and failed, instructively: diet articles mix both kinds of advice together. The heading *"Thực phẩm mẹ bầu cần tránh và nên ăn"* has both cues, and one sentence may say "hạn chế đồ ngọt, thay vào đó nên ăn rau xanh". The output included absurdities like *gestational diabetes AVOIDS green vegetables*.

The final rule makes good-or-bad a **property of the food**, recorded once in the vocabulary: sweets, soft drinks, white rice and refined starch, very sweet fruit (durian, ripe mango, longan, jackfruit), alcohol, fried

food and processed meat are “avoid”, the rest “recommended”. The text then only shows that the food is discussed in a diet section for that type. This matches standard nutrition advice in nearly all cases; it cannot express nuance such as “fruit is fine in moderation”.

#### 4.3.8 Drug side effects

Side effects are scattered across drug sections, complication sections and general articles, so the section rule cannot find them. They are read **within one sentence**: the sentence must contain a drug, a symptom or complication, and a causal cue, *tác dụng phụ* (side effect), *tác dụng không mong muốn* or *gây* (causes):

> “Insulin có thể **gây hạ đường huyết** nếu tiêm quá liều” ⇒ (Insulin, HAS_SIDE_EFFECT, Hạ đường huyết)

Sentences containing *thiếu* (lack of) or *không đủ* are skipped. “*thiếu* insulin gây nhiễm toan ceton” has a drug, a complication and *gây*, yet says that a *lack* of insulin causes ketoacidosis, the reverse of a side effect. Without the exception the graph would accuse insulin of causing the condition it treats. Section 9 shows the filter catches most but not all such sentences.

#### 4.3.9 Background facts added by hand

Some knowledge is not stated under any heading but is general medical classification that articles assume: Metformin is a biguanide, diabetic retinopathy affects the eye, the diagnostic threshold of HbA1c is 6.5%. These are added by hand and labelled `source = curated`, so they can always be separated from extracted facts:

- each drug and its **class** (22 facts, e.g. Metformin → Biguanide, Empagliflozin → Ức chế SGLT2);
- each complication and the **organ** it affects (12 facts);
- each form of diabetes as a **subtype** of diabetes in general (4 facts);
- diagnostic **thresholds** stored on four test nodes, from the Vietnamese Ministry of Health 2020 guideline and the American Diabetes Association criteria that Vinmec also quotes: HbA1c $\geq$ 6.5%; fasting plasma glucose $\geq$ 7.0 mmol/L; 2-hour glucose in the tolerance test $\geq$ 11.1 mmol/L; random glucose $\geq$ 11.1 mmol/L with symptoms.

These few facts enable the most interesting questions. Without “retinopathy affects the eye” the graph can say type 2 causes retinopathy but cannot say which organs type 2 damages.

### 4.4 Output Files

The extractor writes four CSV files:

Table 10: Files produced by the extractor (`dataset/processed/`).

| File | One row is | Purpose |
|---|---|---|
| `entities.csv` | an entity: name, type, threshold | becomes the nodes |
| `triples.csv` | a fact: subject, relation, object, support, source, up to five example URLs | becomes the relationships |
| `articles.csv` | a crawled page: URL, title, page type, date, subject | traces facts back to articles |
| `mentions.csv` | a (page, entity) pair for every entity mentioned anywhere on the page | a looser signal of what each article discusses |

The facts file uses the lecture Exercise 1 form `subject, relation, object` with three extra columns:

```
subject,relation,object,support,source,example_urls
```

```
Đái tháo đường type 2,HAS_SYMPTOM,Khát
 ↪  nhiều,4,vinmec,https://www.vinmec.com/vie/bai-viet/...
Đái tháo đường type
 ↪  2,TREATED_BY,Metformin,11,vinmec,https://www.vinmec.com/vie/bai-viet/...
Metformin,BELONGS_TO_CLASS,Biguanide,0,curated,
Bệnh võng mạc,AFFECTS_ORGAN,Mắt,0,curated,
```

Table 11 shows the best-supported extracted fact of each extracted relation.

Table 11: The best-supported extracted fact of each relation type.

| Subject | Relation | Object | Support |
|---|---|---|---|
| Đái tháo đường | TREATED_BY | Insulin | 143 |
| Đái tháo đường | MANAGED_BY | Tập thể dục đều đặn | 114 |
| Đái tháo đường | HAS_COMPLICATION | Bệnh tim mạch | 56 |
| Đái tháo đường | RECOMMENDS_FOOD | Ngũ cốc nguyên hạt | 54 |
| Đái tháo đường | AVOIDS_FOOD | Rượu bia | 48 |
| Đái tháo đường | DIAGNOSED_BY | HbA1c | 42 |
| Insulin | HAS_SIDE_EFFECT | Hạ đường huyết | 37 |
| Đái tháo đường | HAS_RISK_FACTOR | Thừa cân, béo phì | 27 |
| Đái tháo đường | HAS_SYMPTOM | Mệt mỏi | 25 |

### 4.5 Meeting the Lecture's Requirements

Table 12: Lecture requirements (Mini Project 3, Exercise 1) and what the graph achieves.

| Requirement | Lecture minimum | This project |
|---|---|---|
| Entities | $\geq 50$ | 127 |
| Relationships (facts) | $\geq 100$ | 339 |
| Relationship types | $\geq 5$ | 12 |
| Entity types | $\geq 3$ | 10 |
| Triples stored as CSV | `subject, relation, object` | `triples.csv` (plus support, source, example URLs) |
| Cypher queries | $\geq 5$, including one-hop and two-hop | 15 in `kg/queries.cypher` |
| Visualisation | a drawing of the graph | whole graph and four focused views |
| Graph retrieval | 1–2 hops from an entity | `graph_facts(entity, hops)` |

## 5 System Implementation

### 5.1 Project Structure

The system is written in Python (crawler, extractor, loader, API, notebook) and JavaScript (website). Table 13 lists each part. Every stage reads files or the database and can be rerun alone.

Table 13: Project structure of `graph_01/`.

| Path | Role |
|---|---|
| `kg/crawl.py` | sitemap → candidate URLs → download → relevance filter → `dataset/raw/pages/*.json` and `index.csv`; resumable |
| `kg/vocab.py` | vocabulary (106 entities, 320 aliases), section rules, food polarity, curated classes, organs, thresholds |
| `kg/extract.py` | pages → votes → support filter → `dataset/processed/*.csv` |
| `kg/load.py` | CSV files → Neo4j (wipes and reloads; idempotent) |
| `kg/retriever.py` | `graph_facts(entity, hops)`: bounded graph retrieval |
| `kg/visualize.py` | NetworkX drawings |
| `kg/evaluate.py` | draws the random sample for the quality check |
| `kg/queries.cypher` | the 15 example queries for Neo4j Browser |
| `notebook/01_diabetes_kg.ipynb` | explains and runs every stage, with all figures |
| `api/` | FastAPI service: read-only Cypher behind a small REST interface |
| `web/` | React website for exploring the graph |
| `evaluation/manual_audit.csv` | the 50 sampled facts with the manual judgement |
| `report/` | this report and its plan |

## 5.2 Storing the Graph in Neo4j

### 5.2.1 Why a graph database

A NetworkX graph, as in the lecture's first examples, lives in a Python process's memory: it disappears when the program ends and every program must rebuild it. A real system needs permanent storage shared by the notebook, the website and ad hoc Cypher queries, with quick answers.

Triples could go into a relational table with three columns, but graph questions are about *paths*, and in SQL each step of a path is another self-join: queries grow long and slow as the table grows. A **graph database** keeps direct references from each node to its neighbours, so following an arrow is a jump and not a search. **Neo4j** is the most widely used, and its language **Cypher** writes a path as ASCII art that resembles the drawing. Neo4j 5 runs locally in a Docker container with its data in the project folder.

### 5.2.2 Data model

```
(:Entity:Symptom {name})                          every entity: label Entity + its
↪  own type
(:Entity:Test {name, threshold})
(:DiabetesType)-[:HAS_SYMPTOM {support, source, example_urls}]->(:Symptom)
(:Article {url, title, page_type, date})-[:ABOUT]->(:DiabetesType)
(:Article)-[:MENTIONS]->(:Entity)
```

Three decisions deserve explanation.

**Two labels per entity.** The type label (`Symptom`) allows asking for "all symptoms". The shared `Entity` label carries one rule for all, that names are unique, so there is never a second "Metformin" node, and permits "everything connected to Metformin" without knowing neighbour types.

**Evidence on the relationships.** `support`, `source` and `example_urls` are relationship properties, so one query returns the answer, its documentation and its articles. Stored elsewhere, each answer would need a second lookup.

**Articles are nodes too.** Each of the 1,096 articles is an `Article` node linked by `ABOUT` to its diabetes type and by `MENTIONS` to every entity it mentions anywhere (10,083 links). That preserves the path from clean facts back to the original text and answers "where does this come from?". Since article nodes and `MENTIONS` links are not facts about diabetes, queries about the knowledge restrict themselves to `Entity` nodes.

### 5.2.3 Loading

```
CREATE CONSTRAINT entity_name IF NOT EXISTS FOR (e:Entity) REQUIRE e.name IS
↪  UNIQUE;

UNWIND $rows AS r
MERGE (e:Entity {name: r.name}) SET e:Symptom;          -- one query per entity
↪  type

UNWIND $rows AS r
MATCH (h:Entity {name: r.subject}), (t:Entity {name: r.object})
MERGE (h)-[x:HAS_SYMPTOM]->(t)                           -- one query per
↪  relation type
SET x.support = r.support, x.source = r.source, x.example_urls = r.example_urls;
```

- **`MERGE`, not `CREATE`.** `CREATE` always adds, even if an identical node exists; `MERGE` looks first and creates only if missing. The loader can run repeatedly without duplicating anything.
- **`UNWIND` sends a batch.** Each request has a fixed cost: the 10,083 mention links one by one would be 10,083 round trips, whereas `UNWIND` ships one list and lets the database loop. The graph loads in seconds.
- **The uniqueness constraint is an index** on `name`, so every query that begins at "the node called Metformin" jumps straight to it.
- **Relation types are in the query text.** Cypher cannot take a relationship type or label as a parameter, so the loader groups facts by relation and builds one query per type from the fixed list of twelve.

After loading, the database holds 1,223 nodes (127 entities and 1,096 articles) and 11,518 relationships (339 facts, 10,083 mentions and 1,096 `ABOUT` links).

## 5.3 Querying with Cypher

A Cypher query describes the *shape* of a path: round brackets for nodes, square brackets inside an arrow for relationships.

```
MATCH (t:DiabetesType {name: 'Đái tháo đường type
↪  2'})-[r:HAS_SYMPTOM]->(s:Symptom)
RETURN s.name, r.support
```

Read left to right: find a `DiabetesType` called "type 2", follow outgoing `HAS_SYMPTOM` arrows to `Symptom` nodes, return each name and the arrow's support. Neo4j uses the name index to land on the type 2 node and then examines only that node's arrows, never scanning the rest. Work grows with the number of arrows followed, not with database size. The fifteen queries of `kg/queries.cypher` are presented in Section 7.1.

## 5.4 Graph Retrieval

In a complete question-answering system (the lecture's Knowledge Graph + RAG design) the graph's role is to give a language model a small set of relevant, reliable facts. `kg/retriever.py` implements the lecture's `graph_facts` on Neo4j:

```
ONE_HOP = """
MATCH (e:Entity {name: $name})-[r]-(n:Entity)
RETURN startNode(r).name AS s, type(r) AS rel, endNode(r).name AS o, r.support
↪  AS support
ORDER BY support DESC"""
TWO_HOP = """
```

```
MATCH (e:Entity {name: $name})-[r1]-(m:Entity)-[r2]-(n:Entity) WHERE n <> e
RETURN startNode(r2).name AS s, type(r2) AS rel, endNode(r2).name AS o,
 ↪  r2.support AS support
ORDER BY support DESC"""

def graph_facts(driver, entity, hops=1, limit=30):
    facts, seen = [], set()
    with driver.session() as s:
        for q in [ONE_HOP] + ([TWO_HOP] if hops >= 2 else []):
            for r in s.run(q, name=entity):
                key = (r["s"], r["rel"], r["o"])
                if key not in seen:
                    seen.add(key)
                    facts.append(f"{r['s']} {r['rel']} {r['o']}.")
    return facts[:limit]
```

It starts at the entity named in the question, gathers facts within one or two steps, writes each as a short sentence, ranks by support and returns at most 30.

```
$ python kg/retriever.py "Hạ đường huyết"
Đái tháo đường HAS_COMPLICATION Hạ đường huyết.
Insulin HAS_SIDE_EFFECT Hạ đường huyết.
Đái tháo đường thai kỳ HAS_COMPLICATION Hạ đường huyết.
Metformin HAS_SIDE_EFFECT Hạ đường huyết.
Đái tháo đường type 2 HAS_COMPLICATION Hạ đường huyết.
```

The bound on hops and facts matters because within a few steps almost everything connects to everything, especially through hubs. One step from *Hạ đường huyết* (hypoglycaemia) yields the 5 precise facts above; two steps yield **230**, since one neighbour is the general diabetes node, itself linked to most of the graph. Feeding 230 facts to a language model for a question about hypoglycaemia would bury the relevant ones. The lecture calls this "graph retrieval too broad" (§23.4), and the remedy used here is its recommendation: start at the entity in the question, stay within one or two hops, rank and truncate.

## 5.5 The Website

### 5.5.1 Architecture

Cypher is powerful but not for the typical reader, so a website makes the graph usable by anyone. It has two parts:

- an **API** (`api/`, Python, FastAPI) that accepts simple HTTP requests, runs fixed Cypher against Neo4j and returns JSON;
- a **web client** (`web/`, React 18 with Vite and hand-written CSS, the stack of the course's earlier web applications) shown in the browser.




**Why an API in the middle**, instead of the browser talking to Neo4j directly? Security: the database password stays on the server and the browser can run only the fixed read-only queries the API offers, never arbitrary Cypher. Stability: the site depends on a small documented interface and not on the schema, which can change without breaking it. Simplicity: every request is a plain URL such as `/entities/Metformin`, testable in a browser and documented automatically by FastAPI at `/docs`.

### 5.5.2 API

Table 14: API endpoints.

| Endpoint | Returns |
|---|---|
| `GET /stats` | number of entities per type, facts per relation, articles |
| `GET /entities?q=&type=` | entities whose name contains `q`, prefix matches first, then most connected |
| `GET /entities/{name}?min_support=` | one entity with all its facts (both directions), filtered by support |
| `GET /evidence?subject=&relation=&object=` | the articles that state one fact |
| `GET /entities/{name}/articles` | latest articles that mention the entity |
| `POST /symptom-check` | diabetes types ranked for a list of symptoms |
| `GET /paths?source=&target=` | shortest chains of facts between two entities (at most four steps) |
| `GET /health` | database reachability check |

All queries are parameterised: user input is passed as a parameter (`$name`) and never concatenated into the query, so an odd search string cannot alter it. Each response has a declared shape (a Pydantic model) that validates and documents it. Errors become plain messages: an unknown entity returns `404 No entity named …`, a stopped database `503 The Neo4j database is not reachable`, which the site shows as a banner saying what to start. The entity endpoint, for instance:

```
@app.get("/entities/{name}", response_model=EntityDetail)
def entity(name: str, min_support: int = Query(0, ge=0)):
    rows = db.read(f"""
        MATCH (e:Entity {{name: $name}})
        RETURN e.name AS name, {TYPE.format(v='e')} AS type, e.threshold AS
        ↪  threshold,
               COUNT {{ (:Article)-[:MENTIONS|ABOUT]->(e) }} AS mentioned_in""",
               ↪  name=name)
    if not rows:
        raise HTTPException(404, f"No entity named '{name}'.")
    facts = db.read(f"""
        MATCH (e:Entity {{name: $name}})-[r]-(n:Entity)
        WHERE r.source = 'curated' OR r.support >= $min_support
        WITH startNode(r) AS s, r, endNode(r) AS o
        RETURN s.name AS subject, {TYPE.format(v='s')} AS subject_type, type(r)
        ↪  AS relation,
               o.name AS object, {TYPE.format(v='o')} AS object_type,
               coalesce(r.support, 0) AS support, r.source AS source
        ORDER BY support DESC LIMIT $cap""", name=name, min_support=min_support,
        ↪  cap=config.MAX_FACTS)
    return {**rows[0], "facts": facts}
```

The path finder uses Neo4j's `allShortestPaths` over entity nodes only (articles are excluded, or any two entities would connect through some article mentioning both), with at most four steps. For each step it also reports whether the fact runs forward or backward along the path, so the site can word it correctly ("*is a risk factor for*" and not "*has the risk factor*").

### 5.5.3 Web client

The client has five pages, each with its own address so any view can be bookmarked or shared (for example `#/entity/Metformin` or `#/paths?from=…&to=…`):

1. **Home**: a search box with suggestions as one types, the graph's size, an entry point for each diabetes type, and a short note on how the graph was built;

2. **Entity**: all facts about one entity under plain-language headings, each with its support and a *Sources* button listing the articles; a drawing of its neighbourhood; the latest articles mentioning it;
3. **Diabetes types**: the same page for each type, in tabs, for comparison;
4. **Symptom checker**: pick symptoms and see which types the articles associate with them;
5. **Connections**: pick two entities and see the shortest chains of facts between them.

Several design decisions follow from the evaluation of the graph rather than from looks:

- **Evidence is always visible.** Each fact shows how many articles state it and its sources are one click away; hand-added facts are marked "reference".
- **The reader sets the evidence level.** A selector hides facts stated by fewer than 2, 3, 5 or 10 articles (default 2), which hands the precision/recall trade-off of Section 9.3 to the user instead of fixing it for them.
- **Relations use plain words.** `HAS_COMPLICATION` reads "Complications" from the disease and "Complication of" from the complication; `TREATED_BY` seen from a drug reads "Used to treat".
- **The medical disclaimer appears where it matters**, on the symptom checker: the result reports what the articles say and is not a diagnosis.
- **A recognisable visual identity.** A deep-navy header and hero panel give every page the same frame; a single accent colour marks actions; and each entity type keeps one colour across the whole site (dots, pills, chart segments, group borders, path nodes), so a reader learns the colour code once.
- **Light and dark themes.** All colours are design tokens defined once, with a dark variant selected by the system setting (`prefers-color-scheme`); the entity-type colours are brightened in dark mode so the dots and chart bars stay legible.
- **Every state is handled**: loading, empty results, unknown entity, stopped API or database (a banner with the command to start it). The search box is a standard accessible combobox (arrows, Enter, Escape) and the layout adapts to phones.

Section 8 walks through the site with screenshots.

# 6 Visualisation

A graph of 127 entities and 339 facts is hard to grasp from tables. This section views it from several angles, from the whole network to single neighbourhoods, and explains each picture. All drawings are made with NetworkX from the CSV files loaded into Neo4j, so they show the same facts.

## 6.1 Size of the Graph

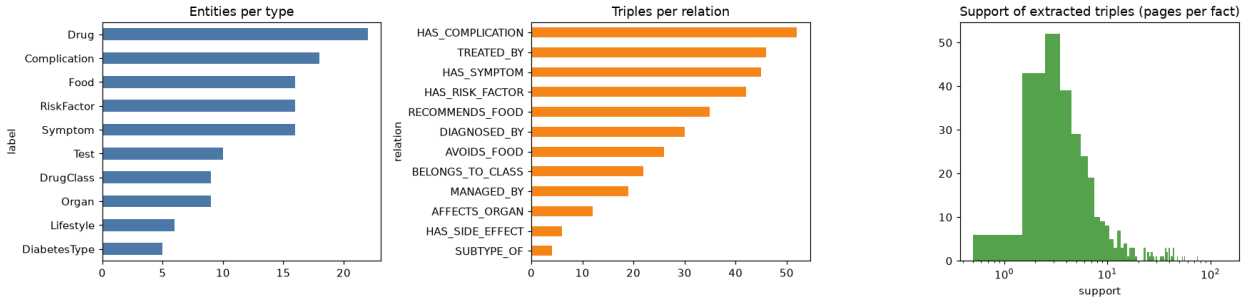



Figure 8: Left: entities per type. Middle: facts per relation type. Right: support of extracted facts (articles per fact), logarithmic scale.

In Figure 8, drugs, complications, risk factors, symptoms and foods each have 16 to 22 entities, about the vocabulary size per type, so nearly every concept found enough support.

The right-hand chart says the most. Most facts rest on a handful of articles (the median support is 5), while a few are repeated by dozens: “diabetes is treated with insulin” by 143 articles, “managed by regular exercise” by 114, “can cause cardiovascular disease” by 56. This long tail is typical of health sites: the core message (exercise, diet, insulin, the main complications) is in nearly every article, while specific facts, such as a newer drug’s class or complications peculiar to pregnancy, appear in a few. Support is thus both a noise filter and a measure of how central a fact is to Vinmec’s way of writing about diabetes.

## 6.2 The Whole Graph

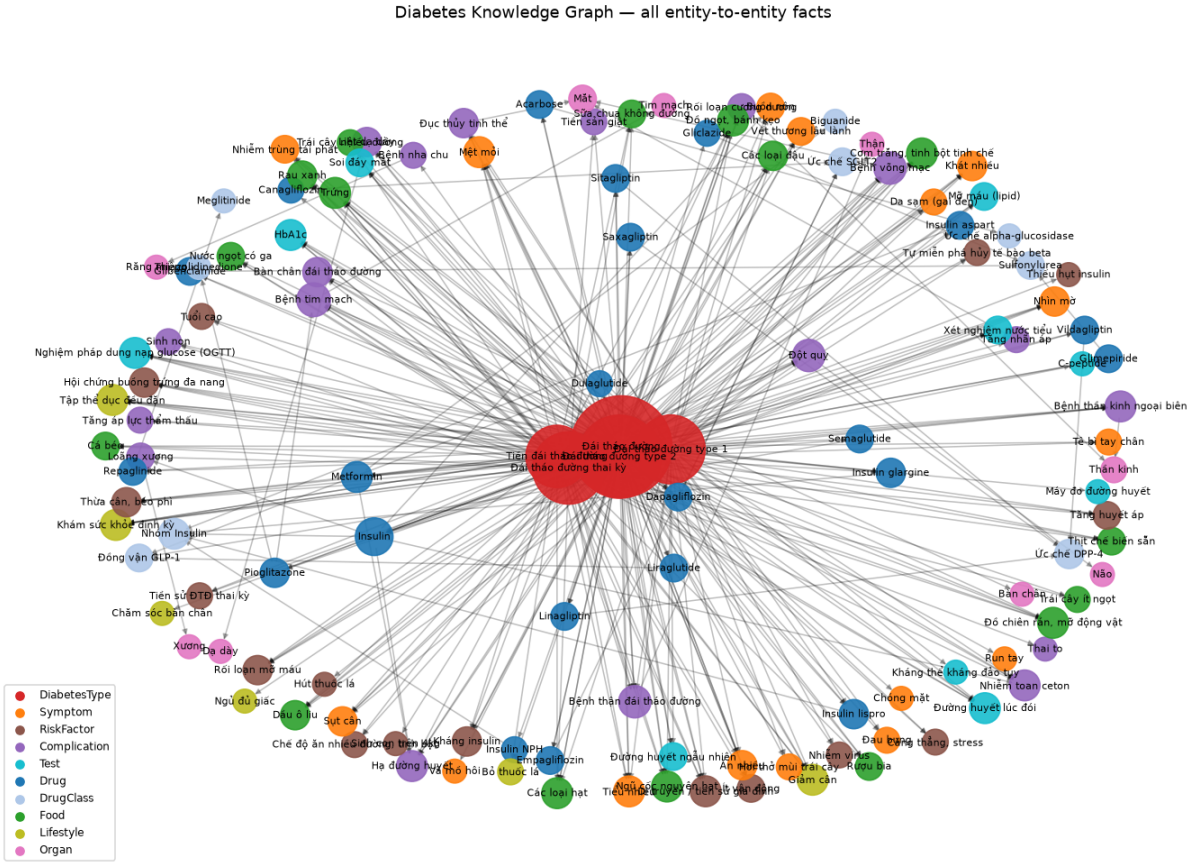



Figure 9: The whole Knowledge Graph: all 339 entity-to-entity facts. Colour is entity type; size is the number of facts the entity is in.

Figure 9 shows the overall structure. The five diabetes types are the big hubs near the middle because almost every extracted fact starts at one; symptoms, complications, drugs, foods and tests surround them, many linked to several types (thirst is a symptom of all). The hand-added facts form short chains at the rim: drug → class, complication → organ.

The graph is **sparse**: of all arrows possible among 127 entities only about 2% exist. Each entity links to a few others, typical of real knowledge and the reason graph queries are fast, since following one node’s arrows touches a small part of the graph.

## 6.3 Focused Views

The whole picture shows structure but is too dense for detail, so three smaller views follow.

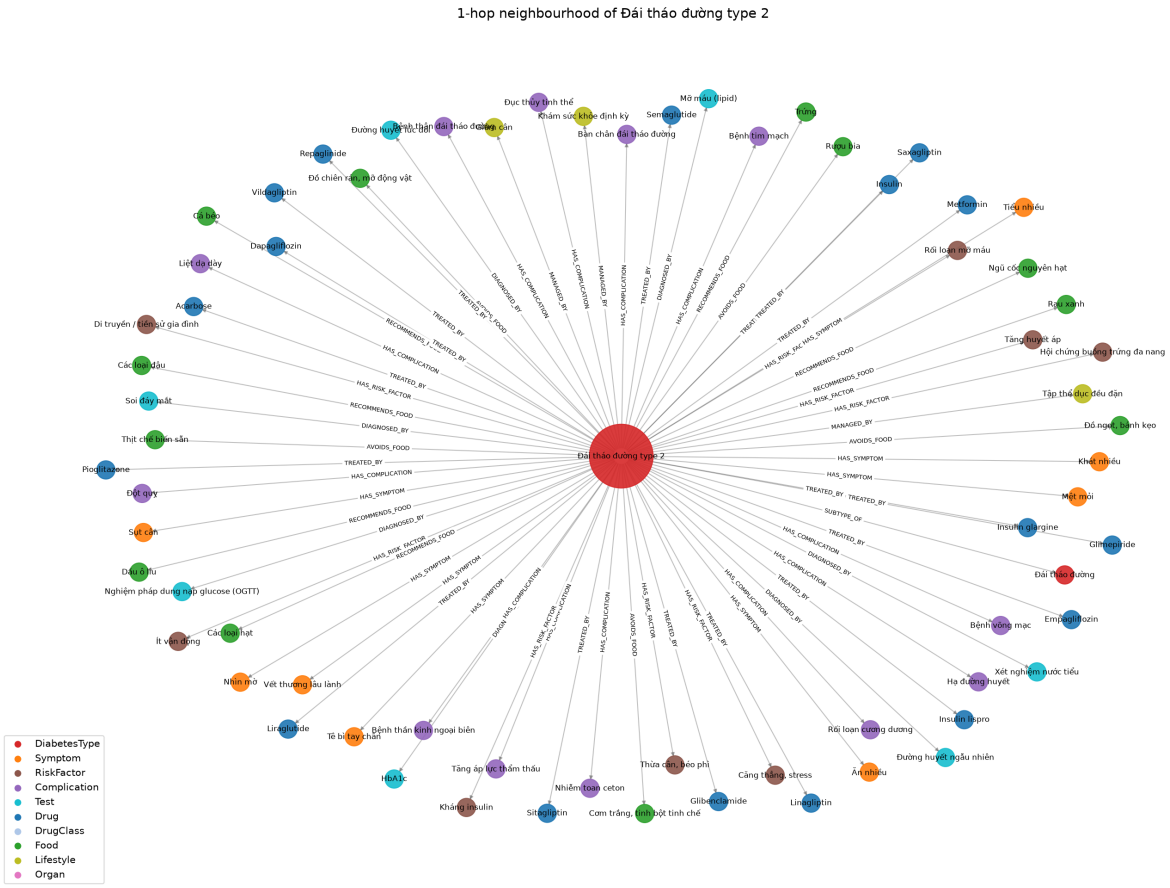



Figure 10: One-hop neighbourhood of *Đái tháo đường type 2*: every fact that touches the type 2 node, with its relation.

Figure 10 is all the graph knows directly about type 2: 17 drugs, 12 foods, 12 complications, 8 risk factors, 8 symptoms, 7 tests and 3 lifestyle measures. It is the picture of a one-hop query.

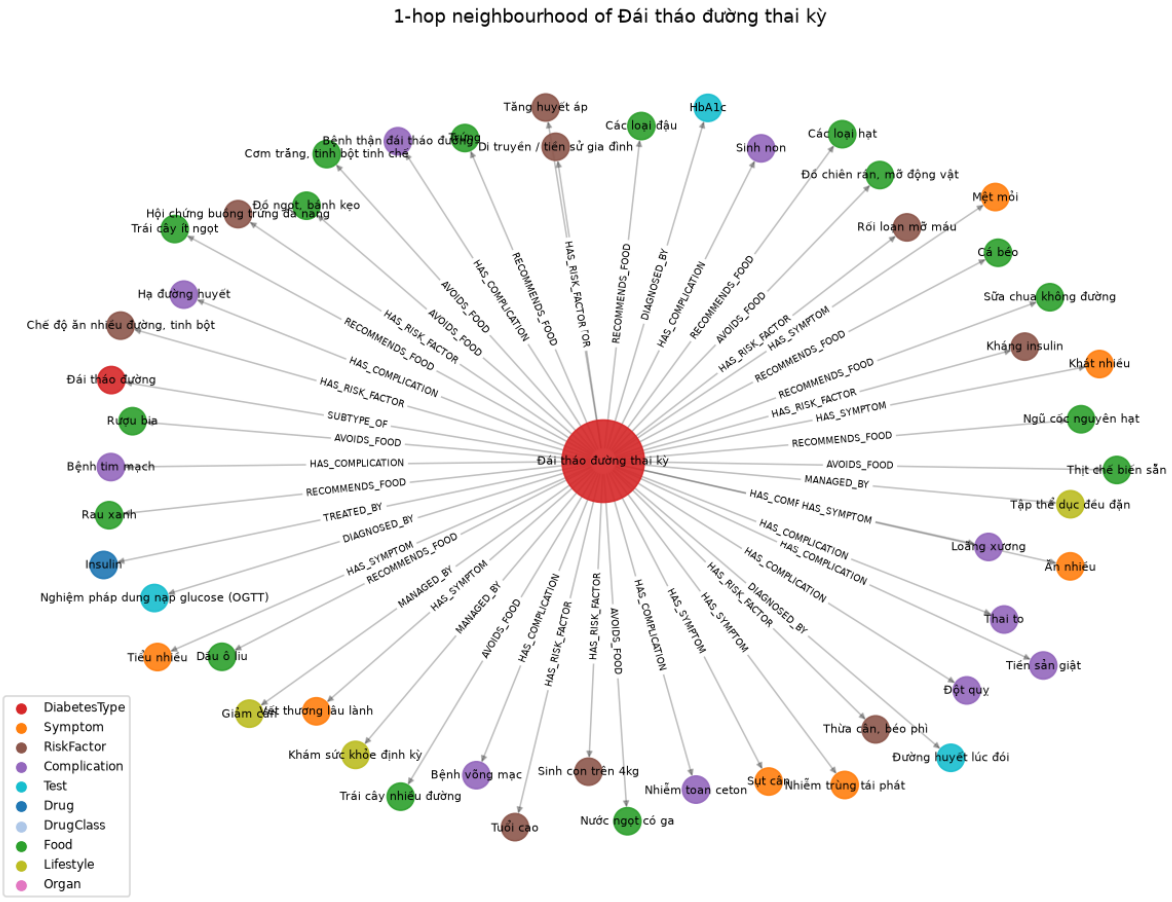



Figure 11: One-hop neighbourhood of *Đái tháo đường thai kỳ* (gestational diabetes).

Figure 11 shows gestational diabetes. Unlike type 2 it has a single drug, insulin (most oral drugs are avoided in pregnancy), and complications of its own: pre-eclampsia, premature birth and an oversized baby. The graph picked up this difference from the articles unaided.

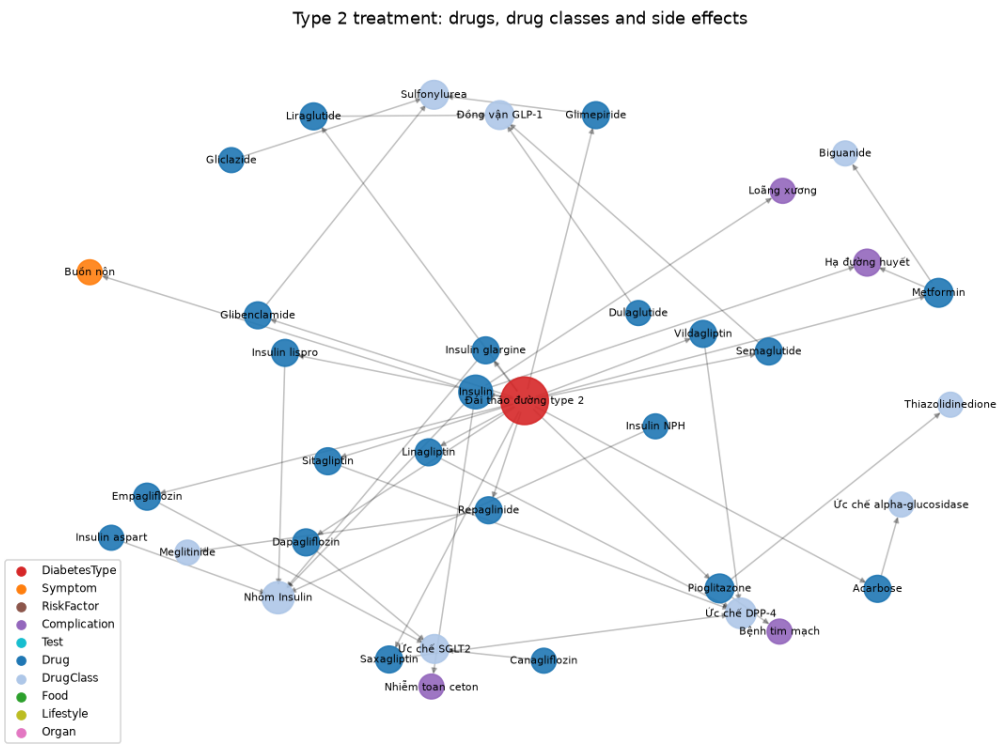



Figure 12: Type 2 treatment: the drugs used, their classes (hand-added) and extracted side effects.

Figure 12 mixes extracted and hand-added facts. The class arrows make drug families cluster: DPP-4 inhibitors around theirs, SGLT2 inhibitors around theirs, the insulin family together, as clinicians group them. The side-effect arrows show the most discussed treatment risk, hypoglycaemia, tied to insulin and Metformin.

## 6.4 Comparing the Diabetes Types

If giving each type its own subject (Section 4.3) works, the types should show different facts. The heatmaps of Figure 13 give, per complication and symptom, the number of supporting articles for each type (darker is more, blank is none).

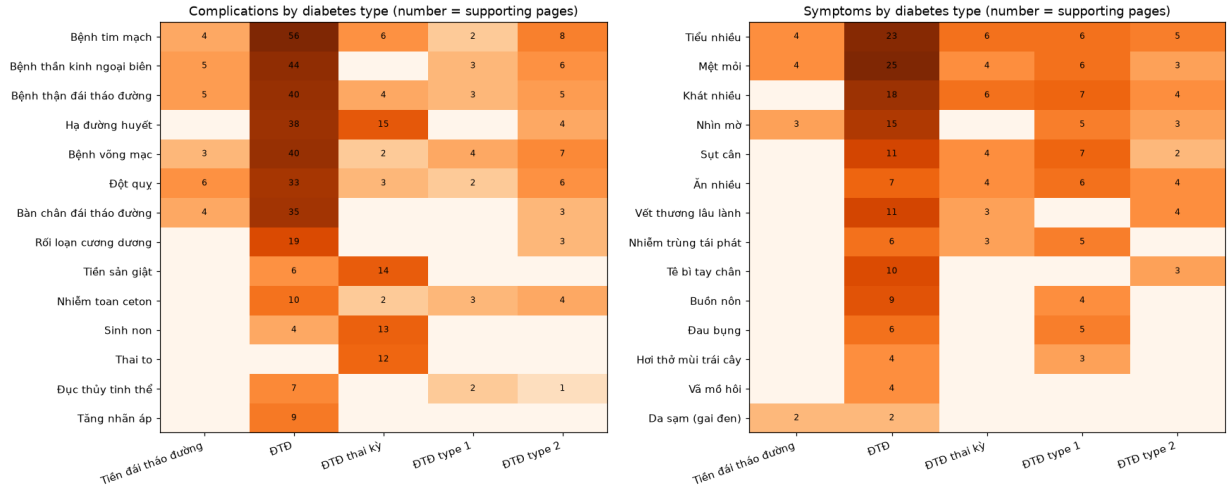



Figure 13: Supporting articles per diabetes type for the most common complications (left) and symptoms (right).

Two things stand out. The general diabetes column is far darker, because most titles name no type. And gestational diabetes has a pattern of its own: hypoglycaemia (of the mother on treatment and of the newborn),

pre-eclampsia, premature birth and a large baby, pregnancy complications the other types lack, while types 1 and 2 share the classic long-term complications of eyes, kidneys, nerves and heart. That is medically right and shows the subject rule captures real differences.

## 6.5 Which Entities Matter Most?

"Important" can mean several things for a node, and two standard measures disagree here, which teaches something about the graph.

**Degree** is the number of facts an entity is in. It looks only at the node's own arrows.

**PageRank**, the method Google first used to rank web pages, holds that a node is important if *important* nodes point to it. Picture a walker who at each step follows a random outgoing arrow of the current node or, 15% of the time, jumps to a random node; after a very long walk, PageRank is the share of time spent on each node.

**What happens here.** All arrows point away from the diabetes types: type → drug → class, type → complication → organ. The walker drifts down these chains and ends at their ends, drug classes and organs, which have no outgoing arrows, and waits for the next jump. Directed PageRank therefore ranks *Nhóm Insulin*, *Ức chế DPP-4* and *Mắt* (eye) first. They are not what the articles are mainly about; they are where paths end.

The arrow direction is our own choice ("type 2 HAS_SYMPTOM thirst" could read "thirst SYMPTOM_OF type 2"), and a score that changes when a relation is renamed does not measure what we want. PageRank is therefore also computed **ignoring direction**, so each arrow can be walked both ways. Then an entity is important if well connected to other well-connected entities, and the ranking is sensible: diabetes types, then insulin, cardiovascular disease, stroke, diabetic kidney disease, retinopathy and Metformin, the treatments and complications that hold the graph together (Figure 14). A graph measure answers one specific question, and one must check it is the intended one.

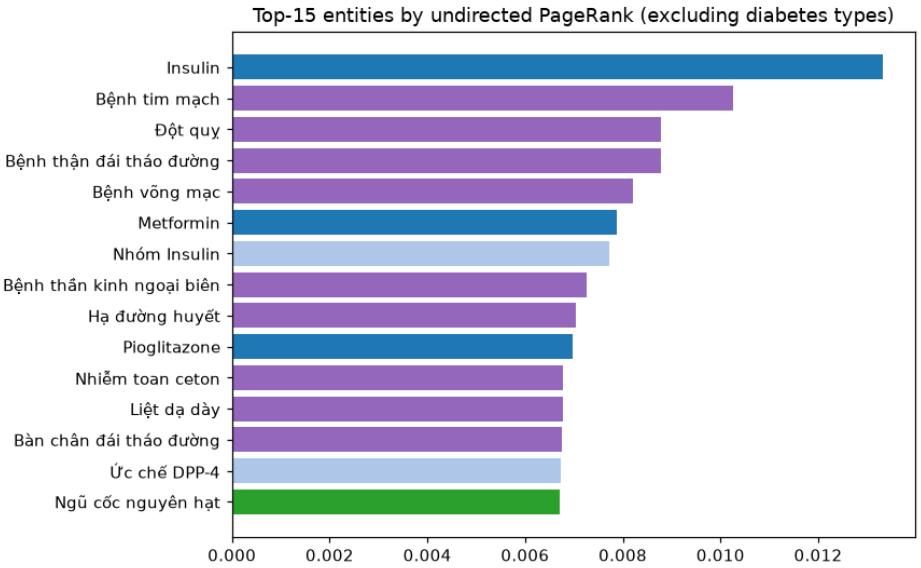



Figure 14: The 15 most central entities by undirected PageRank (diabetes types excluded). Colour is entity type.

## 6.6 What Vinmec Writes About Most

A third view ignores the facts and counts, per entity, the articles that mention it anywhere (`MENTIONS`). This is what the site *talks about*, which differs from connectivity among facts: an entity can be mentioned in many articles in passing, for example when blood sugar is explained, yet appear in few facts under a proper heading.

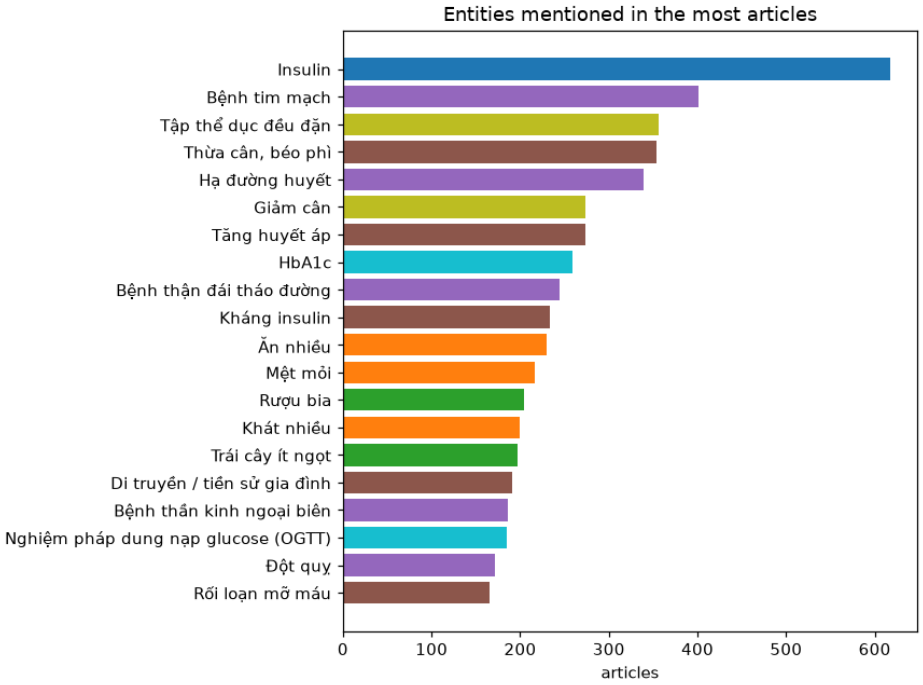



Figure 15: Entities mentioned in the most articles.

# 7 Experimental Evaluation

The lecture insists that an intelligent system be evaluated systematically and not judged by whether it "gives an answer". Two evaluations follow. First, **what the graph can answer**: the example questions of Table 2 are run as Cypher and the results checked for sense. Second, **how correct its facts are**: a random sample of extracted facts is checked by hand against the source articles.

## 7.1 Query Results

The queries come from `kg/queries.cypher`, which holds fifteen (the lecture requires five). Each is shown with its result, taken directly from the database, and a short reading.

### 7.1.1 One-hop: everything about Metformin

```
MATCH (e:Entity {name: 'Metformin'})-[r]-(n:Entity)
RETURN startNode(r).name AS subject, type(r) AS relation, endNode(r).name AS
↪  object,
       r.support AS support, r.source AS source
ORDER BY support DESC
```

Table 15: One-hop neighbourhood of Metformin.

| Subject | Relation | Object | Support | Source |
|---|---|---|---|---|
| Đái tháo đường | TREATED_BY | Metformin | 37 | vinmec |
| Đái tháo đường type 2 | TREATED_BY | Metformin | 11 | vinmec |
| Tiền đái tháo đường | TREATED_BY | Metformin | 8 | vinmec |
| Metformin | HAS_SIDE_EFFECT | Hạ đường huyết | 5 | vinmec |
| Metformin | BELONGS_TO_CLASS | Biguanide | 0 | curated |

The pattern `-[r]-` has no arrowhead and matches both directions, so the result holds facts pointing *to* Metformin (diabetes *is treated with* it) and the one pointing *from* it (it *belongs to* the biguanide class). For

"what do we know about X" both are needed. The support column shows solidity: 37 articles say diabetes is treated with Metformin, 11 name type 2, 8 mention it for prediabetes (where it delays onset of type 2), and 5 cite hypoglycaemia as a risk. The class has support 0 because it was added by hand.

### 7.1.2 Two-hop: which organs can type 2 diabetes damage?

```
MATCH (t:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_COMPLICATION]->(c)
      -[:AFFECTS_ORGAN]->(o:Organ)
RETURN o.name AS organ, collect(c.name) AS via_complications, sum(r.support) AS
↪  evidence
ORDER BY evidence DESC
```

Table 16: Organs reached from type 2 diabetes in two hops.

| Organ | Via complications | Evidence (articles) |
|---|---|---|
| Thần kinh | Rối loạn cương dương, Bệnh thần kinh ngoại biên | 9 |
| Mắt | Đục thủy tinh thể, Bệnh võng mạc | 8 |
| Tim mạch | Bệnh tim mạch | 8 |
| Não | Đột quỵ | 6 |
| Thận | Bệnh thận đái tháo đường | 5 |
| Bàn chân | Bàn chân đái tháo đường | 3 |
| Dạ dày | Liệt dạ dày | 1 |

No article states "type 2 diabetes damages the nerves" in this form. The answer exists only because two kinds of facts meet at the complication node: an extracted one (type 2 *can cause* peripheral neuropathy) and a hand-added one (neuropathy *affects* the nerves). `collect()` groups complications leading to the same organ so each organ appears once with its reasons. The list matches the textbook set of organs harmed by diabetes: nerves, eyes, heart and vessels, brain (stroke), kidneys, feet and stomach.

### 7.1.3 Comparing type 1 and type 2 symptoms

```
MATCH (s:Symptom)
OPTIONAL MATCH (:DiabetesType {name: 'Đái tháo đường type
↪  1'})-[a:HAS_SYMPTOM]->(s)
OPTIONAL MATCH (:DiabetesType {name: 'Đái tháo đường type
↪  2'})-[b:HAS_SYMPTOM]->(s)
WITH s, a, b WHERE a IS NOT NULL OR b IS NOT NULL
RETURN s.name, a.support AS type1, b.support AS type2
```

Table 17: Symptoms of type 1 and type 2 with supporting articles for each.

| Symptom | Type 1 | Type 2 | Found in |
|---|---|---|---|
| Khát nhiều | 7 | 4 | both |
| Mệt mỏi | 6 | 3 | both |
| Nhìn mờ | 5 | 3 | both |
| Sụt cân | 7 | 2 | both |
| Tiểu nhiều | 6 | 5 | both |
| Ăn nhiều | 6 | 4 | both |
| Buồn nôn | 4 | – | type 1 |
| Hơi thở mùi trái cây | 3 | – | type 1 |
| Nhiễm trùng tái phát | 5 | – | type 1 |
| Đau bụng | 5 | – | type 1 |
| Tê bì tay chân | – | 3 | type 2 |
| Vết thương lâu lành | – | 4 | type 2 |

Six symptoms are shared (thirst, frequent urination, hunger, weight loss, fatigue, blurred vision): they come from high blood sugar itself, whatever the cause, so they should be. Most type 1-only symptoms (nausea, abdominal pain, fruity breath) are signs of **ketoacidosis**, the acid build-up when insulin is nearly absent, typical when type 1 is first found. The type 2-only ones (slow-healing wounds, numb hands and feet) arise after years of moderately raised sugar, matching type 2's slow, often unnoticed course. The graph reproduces a real clinical difference without being told. (Recurrent infections appear only for type 1 here; they are a general sign of high sugar that the type 2 articles did not list under a symptom heading.)

### 7.1.4 A symptom checker

```
WITH ['Khát nhiều', 'Tiểu nhiều', 'Sụt cân', 'Nhìn mờ'] AS input
MATCH (t:DiabetesType)-[r:HAS_SYMPTOM]->(s:Symptom) WHERE s.name IN input
RETURN t.name AS type, count(s) AS matched, collect(s.name) AS symptoms,
       sum(r.support) AS evidence
ORDER BY matched DESC, evidence DESC
```

Table 18: Diabetes types ranked for {thirst, frequent urination, weight loss, blurred vision}.

| Type | Matched | Symptoms | Evidence |
|---|---|---|---|
| Đái tháo đường | 4 | Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều | 67 |
| Đái tháo đường type 1 | 4 | Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều | 25 |
| Đái tháo đường type 2 | 4 | Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều | 14 |
| Đái tháo đường thai kỳ | 3 | Khát nhiều, Sụt cân, Tiểu nhiều | 16 |
| Tiền đái tháo đường | 2 | Nhìn mờ, Tiểu nhiều | 7 |

Types are ranked by how many of the given symptoms they have, ties broken by total supporting articles, i.e. how strongly the articles tie the type to these symptoms. General diabetes, type 1 and type 2 match all four, with general diabetes first only because most articles are general. The result shows use and limit together: it says which types the articles associate with the symptoms, but classic symptoms are shared by all types, so it cannot tell them apart. Diagnosis needs blood tests, which the next query shows.

### 7.1.5 Diagnosis: tests and thresholds

```
MATCH (t:DiabetesType)-[r:DIAGNOSED_BY]->(x:Test)
RETURN x.name AS test, x.threshold AS threshold, sum(r.support) AS evidence
ORDER BY evidence DESC
```

Table 19: Diagnostic tests, their thresholds (hand-added node property) and supporting articles (extracted).

| Test | Threshold | Evidence |
|---|---|---|
| HbA1c | $\geq$ 6.5% | 67 |
| Nghiệm pháp dung nạp glucose (OGTT) | 2 giờ $\geq$ 11.1 mmol/L (200 mg/dL) | 59 |
| Đường huyết lúc đói | $\geq$ 7.0 mmol/L (126 mg/dL) | 51 |
| Xét nghiệm nước tiểu | – | 14 |
| Soi đáy mắt | – | 11 |
| Đường huyết ngẫu nhiên | $\geq$ 11.1 mmol/L kèm triệu chứng | 11 |
| Máy đo đường huyết | – | 10 |
| Mỡ máu (lipid) | – | 7 |
| C-peptide | – | 2 |
| Kháng thể kháng đảo tụy | – | 1 |

The answer joins the two kinds of knowledge. "Diabetes is diagnosed with HbA1c" was extracted; "6.5% or more" is a node property added from the official guideline. The three best-evidenced tests, HbA1c, the

glucose tolerance test and fasting glucose, are exactly the three standard criteria. Tests with little evidence are specialised: islet antibodies and C-peptide help tell type 1 from type 2.

### 7.1.6 Diet for gestational diabetes

Table 20: Foods discussed in diet sections of gestational-diabetes articles.

| Advice | Food | Articles |
|---|---|---|
| avoid | Đồ ngọt, bánh kẹo | 14 |
| avoid | Thịt chế biến sẵn | 7 |
| avoid | Nước ngọt có ga | 7 |
| avoid | Rượu bia | 6 |
| avoid | Đồ chiên rán, mỡ động vật | 6 |
| avoid | Trái cây nhiều đường | 4 |
| avoid | Cơm trắng, tinh bột tinh chế | 2 |
| recommended | Rau xanh | 13 |
| recommended | Trứng | 12 |
| recommended | Ngũ cốc nguyên hạt | 10 |
| recommended | Các loại đậu | 8 |
| recommended | Sữa chua không đường | 7 |
| recommended | Các loại hạt | 6 |
| recommended | Trái cây ít ngọt | 5 |
| recommended | Cá béo | 2 |
| recommended | Dầu ô liu | 2 |

Sweets are the food to avoid that is discussed most; green vegetables, eggs and whole grains lead the recommended foods, consistent with usual advice for pregnant women with diabetes. Remember that the avoid/recommend split comes from the fixed food list (Section 4.3); what the articles contribute is *which* foods are discussed for this type and how often.

### 7.1.7 Provenance: where does a fact come from?

```
MATCH (:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_COMPLICATION]->
      (:Complication {name: 'Bệnh thận đái tháo đường'})
UNWIND r.example_urls AS url
MATCH (a:Article {url: url})
RETURN a.title
```

Table 21: Articles stating that type 2 diabetes can cause diabetic kidney disease.

| Article title |
|---|
| Bệnh tiểu đường type 2 có nguy hiểm không? |
| Bệnh tiểu đường type 2: Nguyên nhân, biến chứng và phòng bệnh |
| Các biến chứng nguy hiểm của bệnh đái tháo đường type 2 |
| Cách tự nhiên điều trị bệnh tiểu đường type 2 với những thực phẩm hàng ngày |
| Triệu chứng sớm của đái tháo đường type 2 |

Each fact stores up to five example URLs; joining them with the article nodes returns real Vinmec titles that anyone can open and read. This is what makes a graph trustworthy on health topics: a language model can answer fluently and wrongly with no way to check, while here every answer leads to its source.

### 7.1.8 Co-occurrence through articles

Table 22: Pairs of complications mentioned together in the most articles.

| Complication | Complication | Articles |
|---|---|---|
| Bệnh tim mạch | Hạ đường huyết | 170 |
| Bệnh thận đái tháo đường | Bệnh tim mạch | 163 |
| Bệnh tim mạch | Đột quỵ | 139 |
| Bệnh thận đái tháo đường | Hạ đường huyết | 116 |
| Bệnh thần kinh ngoại biên | Bệnh tim mạch | 110 |
| Bệnh thận đái tháo đường | Bệnh võng mạc | 107 |
| Bệnh thần kinh ngoại biên | Bệnh thận đái tháo đường | 100 |
| Bệnh tim mạch | Bệnh võng mạc | 98 |

The `MENTIONS` links give a looser signal: which topics share articles. Co-appearance is not a fact about diabetes, but it shows how Vinmec writes about it. Cardiovascular disease and hypoglycaemia lead simply because each is mentioned in a large share of all articles, the known weakness of raw co-occurrence: pairs of common topics always rank high. Finding *unusually* frequent pairs would require comparing each count with what each topic's own frequency predicts.

## 7.2 Correctness of the Facts

### 7.2.1 What was measured and how

The measure is **precision**: of the facts in the graph, what share is correct. It was chosen because it is what a user experiences: every wrong fact is something the system states with confidence. The procedure (`kg/evaluate.py`):

1. draw 50 extracted facts at random with a fixed seed (42), so the sample can be redrawn;
2. for each, show the article title and an evidence sentence from a section the extractor actually used;
3. judge the fact by reading **all** its supporting articles, not only the sentence shown, and record `correct` and an `error_type` in `evaluation/manual_audit.csv`.

A fact counts as correct when the Vinmec articles state it for that form of diabetes. This checks against the source, not against clinical guidelines, which would be a different and costlier medical review. Random, because hand-picked facts would unconsciously be easy ones and flatter the result; fifty, because each needs several articles read, and fifty suffices to estimate precision usefully and reveal the main error kinds.

Table 23: The first 12 of the 50 sampled facts with their judgement. Support is the value at audit time, before the stricter heading rule of Section 9.

| # | Subject | Relation | Object | Support | Correct |
|---|---|---|---|---|---|
| 1 | Tiền đái tháo đường | HAS_COMPLICATION | Bệnh thần kinh ngoại biên | 5 | yes |
| 2 | Đái tháo đường thai kỳ | AVOIDS_FOOD | Thịt chế biến sẵn | 7 | yes |
| 3 | Đái tháo đường thai kỳ | HAS_RISK_FACTOR | Chế độ ăn nhiều đường, tinh bột | 2 | yes |
| 4 | Đái tháo đường | HAS_RISK_FACTOR | Rối loạn mỡ máu | 16 | **no** |
| 5 | Tiền đái tháo đường | HAS_RISK_FACTOR | Tuổi cao | 2 | yes |
| 6 | Đái tháo đường | HAS_COMPLICATION | Tăng nhãn áp | 9 | yes |
| 7 | Đái tháo đường type 2 | DIAGNOSED_BY | Đường huyết ngẫu nhiên | 4 | yes |
| 8 | Đái tháo đường | TREATED_BY | Saxagliptin | 5 | yes |
| 9 | Đái tháo đường type 1 | DIAGNOSED_BY | Xét nghiệm nước tiểu | 2 | yes |
| 10 | Đái tháo đường type 2 | TREATED_BY | Glimepiride | 2 | yes |
| 11 | Đái tháo đường | MANAGED_BY | Ngủ đủ giấc | 7 | yes |
| 12 | Đái tháo đường thai kỳ | AVOIDS_FOOD | Trái cây nhiều đường | 4 | yes |

### 7.2.2 Result

Table 24: Precision on the 50 sampled facts, by relation type.

| Relation | Sampled | Correct |
|---|---|---|
| AVOIDS_FOOD | 7 | 7 |
| DIAGNOSED_BY | 6 | 6 |
| HAS_COMPLICATION | 7 | 7 |
| HAS_RISK_FACTOR | 11 | 10 |
| HAS_SYMPTOM | 7 | 6 |
| MANAGED_BY | 2 | 2 |
| RECOMMENDS_FOOD | 3 | 3 |
| TREATED_BY | 7 | 7 |
| **All** | 50 | 48 |

**48 of 50 facts are correct: precision 0.96.** Both errors fall in symptoms and risk factors, the two relation types most dependent on the heading; every sampled complication, test, drug and food fact was correct.

How precise is "0.96"? On 50 facts the true figure for the whole graph may be higher or lower: another sample might hold one error or four. The Wilson interval for a proportion on a small sample gives a likely range of **0.87 to 0.99**. In plain terms, roughly nine facts in ten or better are right, and "96%" is not an exact figure. Section 9 analyses the two errors and the response.

## 7.3 How Much Evidence the Facts Have

Table 25: Extracted facts remaining when a minimum number of supporting articles is required.

| Minimum support | Facts | Share |
|---|---|---|
| $\geq 1$ | 301 | 100% |
| $\geq 2$ | 295 | 98% |
| $\geq 3$ | 252 | 84% |
| $\geq 5$ | 161 | 53% |
| $\geq 10$ | 70 | 23% |

Table 25 shows the graph shrinking as more evidence is demanded. The six facts at support 1 come from Vinmec's disease pages (Section 4.3). Requiring three articles keeps 84% of facts, requiring ten keeps 23%, the core facts Vinmec repeats across many articles. Because support is stored on every fact, the choice need not be made once for everyone: a query, or the website's user, picks the level that suits the purpose.

# 8 Demo

This section follows the website as a user would, with screenshots of the running system. Each scenario matches a question of Table 2, and all screenshots were taken from the live site connected to the database described above.

## 8.1 Scenario 1: Finding an Entity

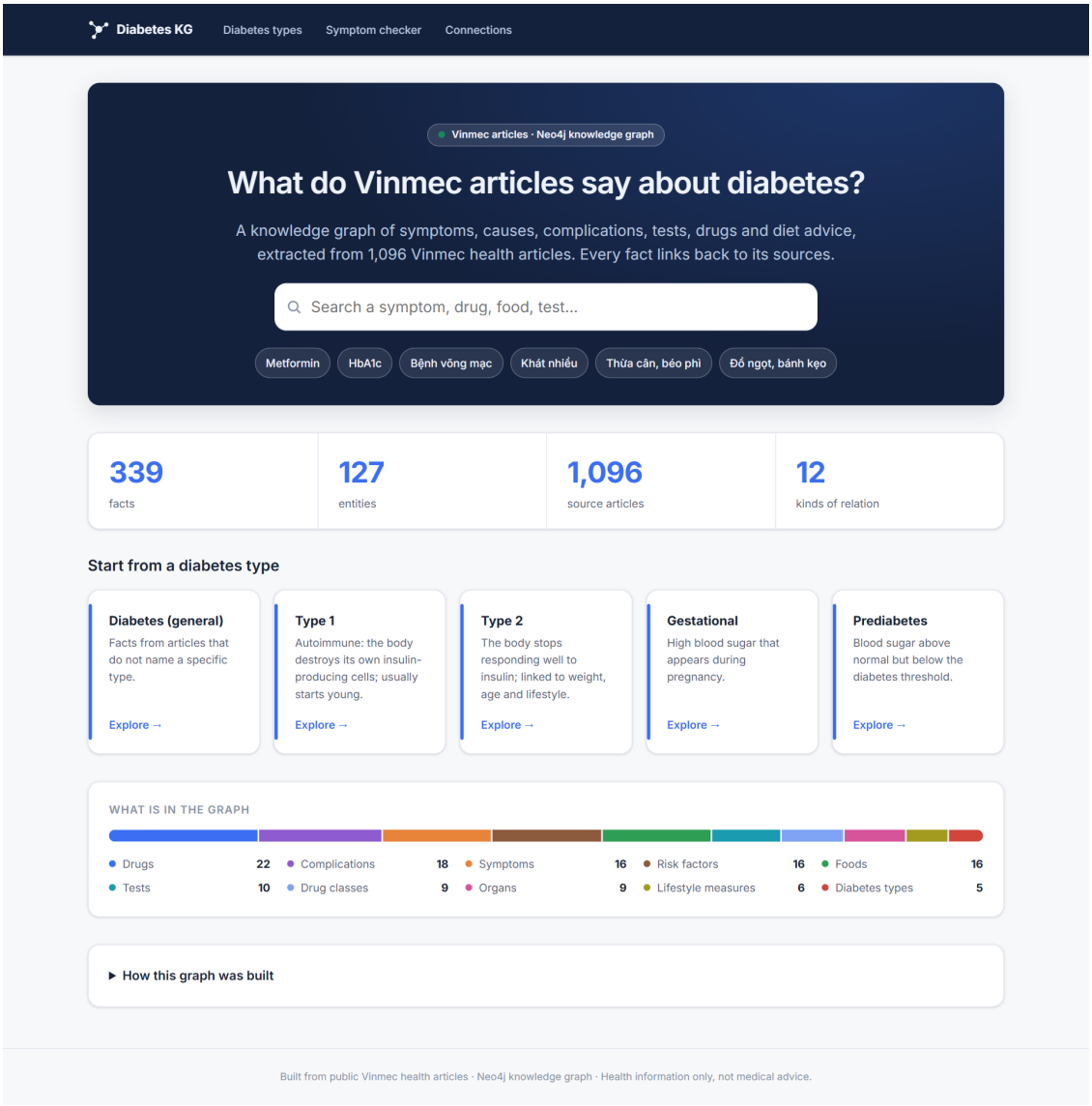



Figure 16: Home page: search box and quick links in the header panel, the four size figures, an entry card for each diabetes type, and a bar showing what the graph is made of.

The home page (Figure 16) opens with a navy hero panel that says in one sentence what the site is, what Vinmec articles say about diabetes as a graph built from 1,096 articles, and offers three ways in: search, an example entity, or a diabetes type. A single strip summarises the graph: 339 facts, 127 entities, 1,096 source articles and 12 kinds of relation. Below it, one card per diabetes type (each with a one-line explanation and an *Explore* cue) leads into the type pages, and a stacked bar with a legend shows what the 127 entities are made of (22 drugs, 18 complications, 16 symptoms, 16 risk factors, 16 foods, 10 tests, 9 drug classes, 9 organs, 6 lifestyle measures and 5 diabetes types), so a newcomer sees the scope at a glance. A collapsible note explains in four steps how the graph was built and states its measured precision and that it is not medical advice.

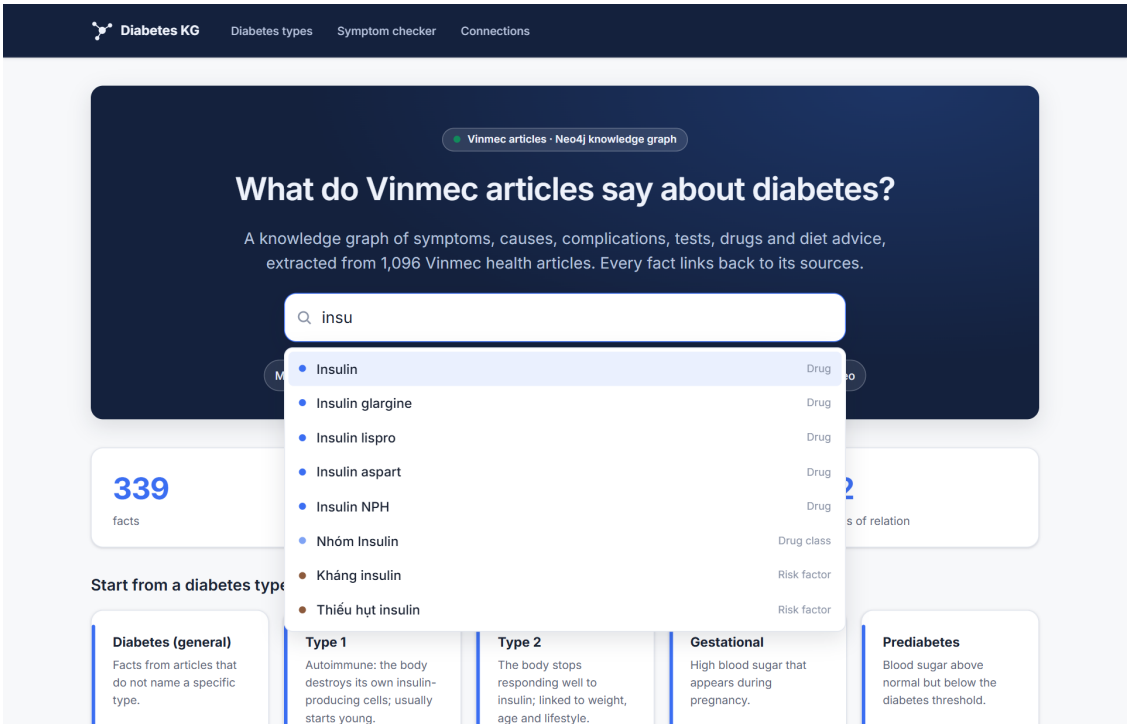



Figure 17: Typing “insu”: suggestions appear as one types, each with its type.

Typing “insu” (Figure 17) at once lists entities whose names contain those letters: the insulin drugs, the insulin drug class, and risk factors such as insulin resistance. A coloured dot shows each type, so *Kháng insulin* is not mistaken for a drug. Names that *begin* with the typed letters come first, then the most connected. The list works with the keyboard (arrows, Enter), as a standard search field should, and each keystroke cancels the previous request so fast typing never shows stale results.

## 8.2 Scenario 2: Everything About One Entity, With Sources

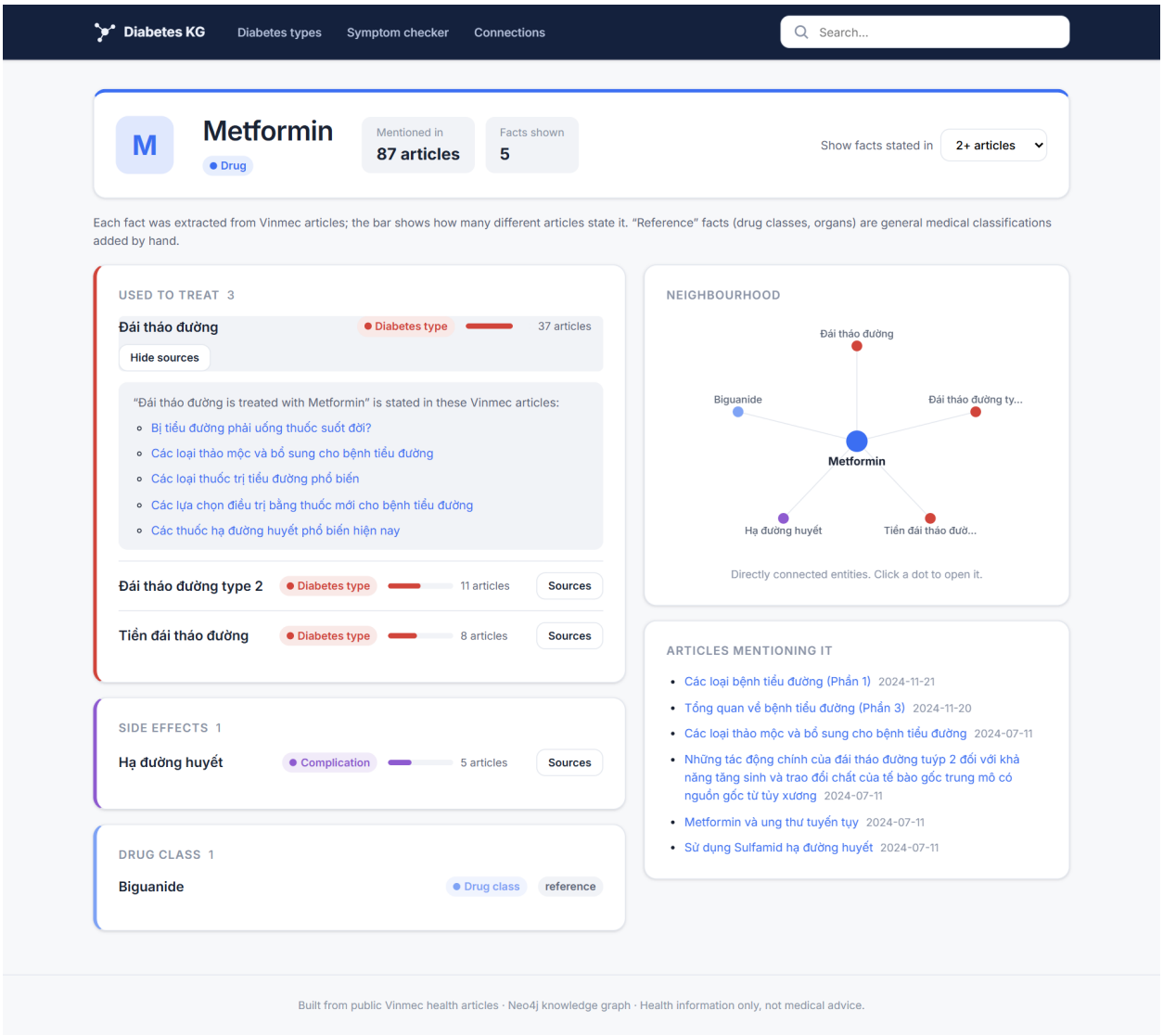



Figure 18: The Metformin page: facts grouped by relation with their supporting-article counts, sources of the first fact open; on the right the neighbourhood drawing and the latest articles mentioning Metformin.

Selecting *Metformin* opens its page (Figure 18). A header card, topped by a border in the colour of the entity's type and carrying an initial-letter avatar, shows the name, the type, the number of articles mentioning it (87) and the number of facts shown. Facts sit under plain-language headings derived from relation and direction: *Used to treat* (diabetes, type 2, prediabetes), *Side effects* (hypoglycaemia), *Drug class* (biguanide). Each group has a left border and bars in the colour of the entity type it lists (red for diabetes types, purple for complications), which makes the groups easy to tell apart. Each fact has a bar and a number giving how many different articles state it; the bar is logarithmic so that 5 articles stay visible next to 37, but the number is always shown because the bar is decoration.

*Sources* opens the evidence for one fact, with links to up to five articles. This is the provenance query of Section 7.1 offered to any reader. The drug class is labelled "reference" and has no bar because it was added by hand.

The drawing on the right is the entity's one-hop neighbourhood (up to its twelve strongest neighbours), coloured by type; every dot links to that entity's page, so one can explore by clicking from entity to entity. Beneath it, the latest articles mentioning Metformin are listed with dates. The selector *Show facts stated in* (default 2+ articles) lets the reader decide how much evidence a fact needs, the precision/recall trade-off of Section 9.3 handed to the user.

## 8.3 Scenario 3: Comparing the Forms of Diabetes

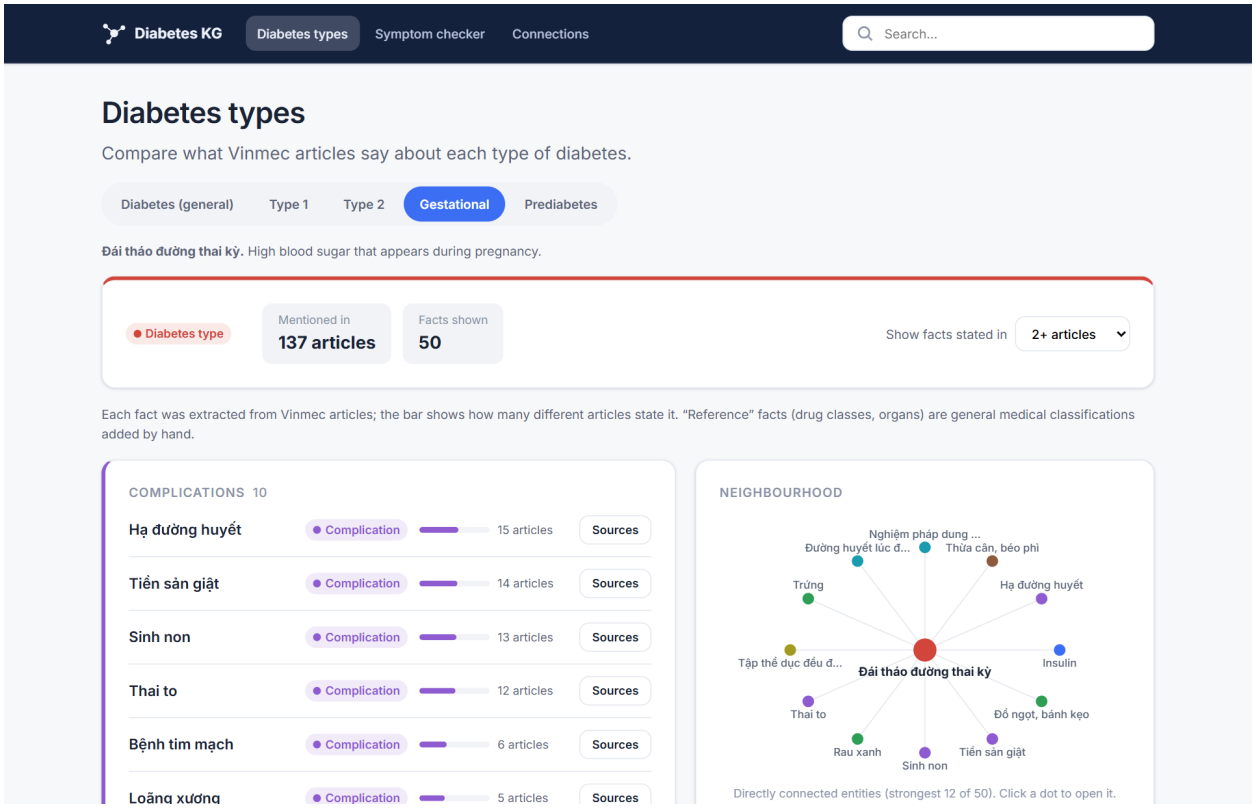



Figure 19: The top of the *Diabetes types* page with the gestational tab open (the page continues with the foods, risk factors and tests of that type).

The *Diabetes types* page (Figure 19) puts the five types in a segmented control, each with a one-line plain explanation ("High blood sugar that appears during pregnancy"). Switching tabs shows how facts differ. For gestational diabetes the complications start with hypoglycaemia (15 articles), pre-eclampsia (14), premature birth (13) and a large baby (12), the pregnancy-specific pattern of the heatmap in Figure 13, and the neighbourhood drawing shows its foods, tests and risk factors at a glance.

## 8.4 Scenario 4: Checking Symptoms

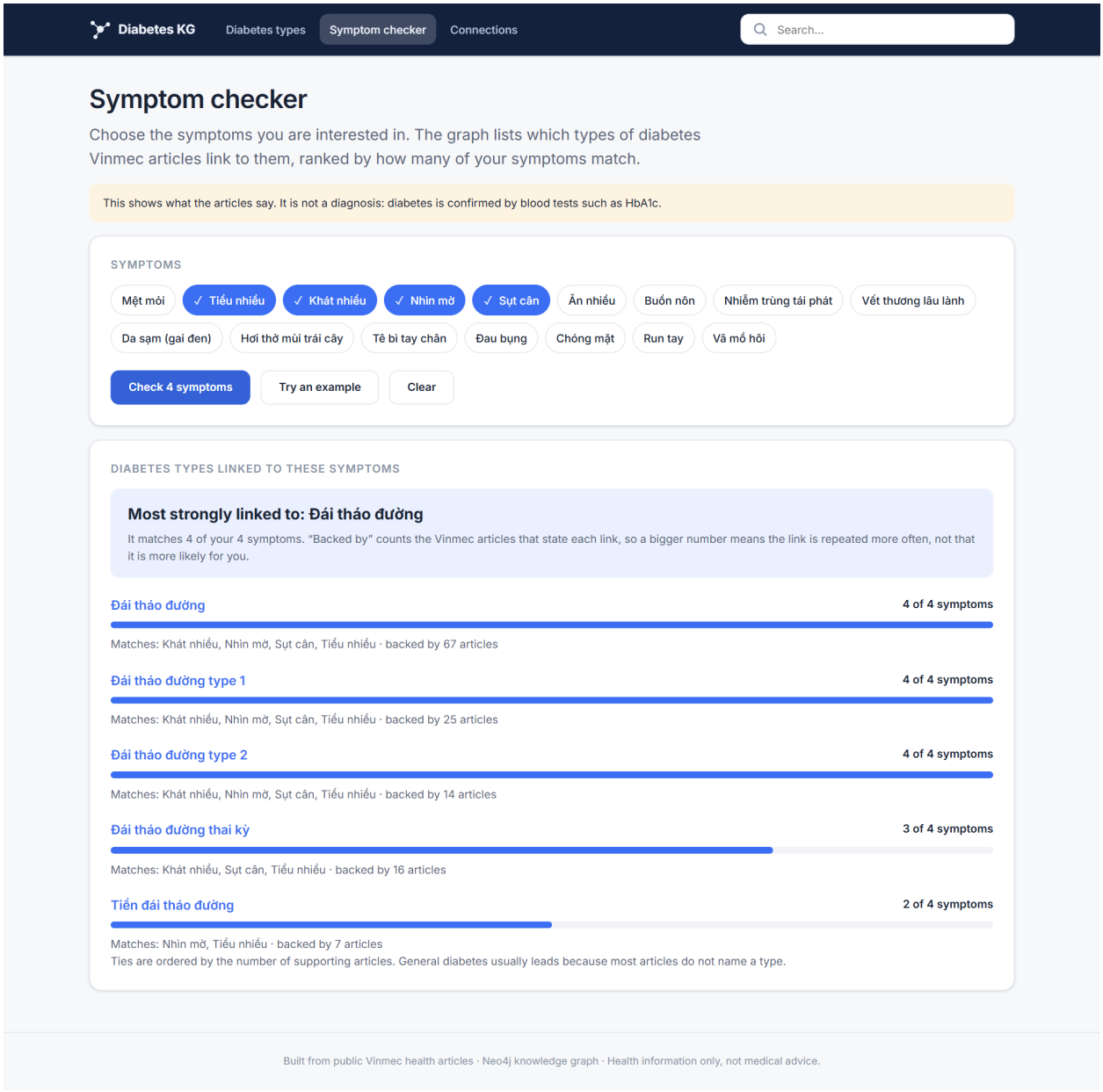



Figure 20: The symptom checker with four symptoms chosen and the ranked result.

On the symptom checker (Figure 20) the user picks from all 16 symptom entities and presses *Check*. The result opens with a plain-language verdict ("Most strongly linked to: Đái tháo đường"), followed by a note on how to read the numbers: "backed by" counts the Vinmec articles that state each link, so a larger value means the link is repeated more often, not that it is more likely for the user. Below it the types are ranked by how many chosen symptoms the articles tie to them ("4 of 4 symptoms"), with a bar for the share matched, the matches, and the supporting-article count. A *Try an example* button fills in the four symptoms of the figure and runs the check in one click. A notice states that this shows what the articles say and is not a diagnosis, since diabetes is confirmed by blood tests such as HbA1c. It reproduces the Cypher query of Table 18, in a form usable without knowing the graph.

## 8.5 Scenario 5: How Are Two Things Connected?

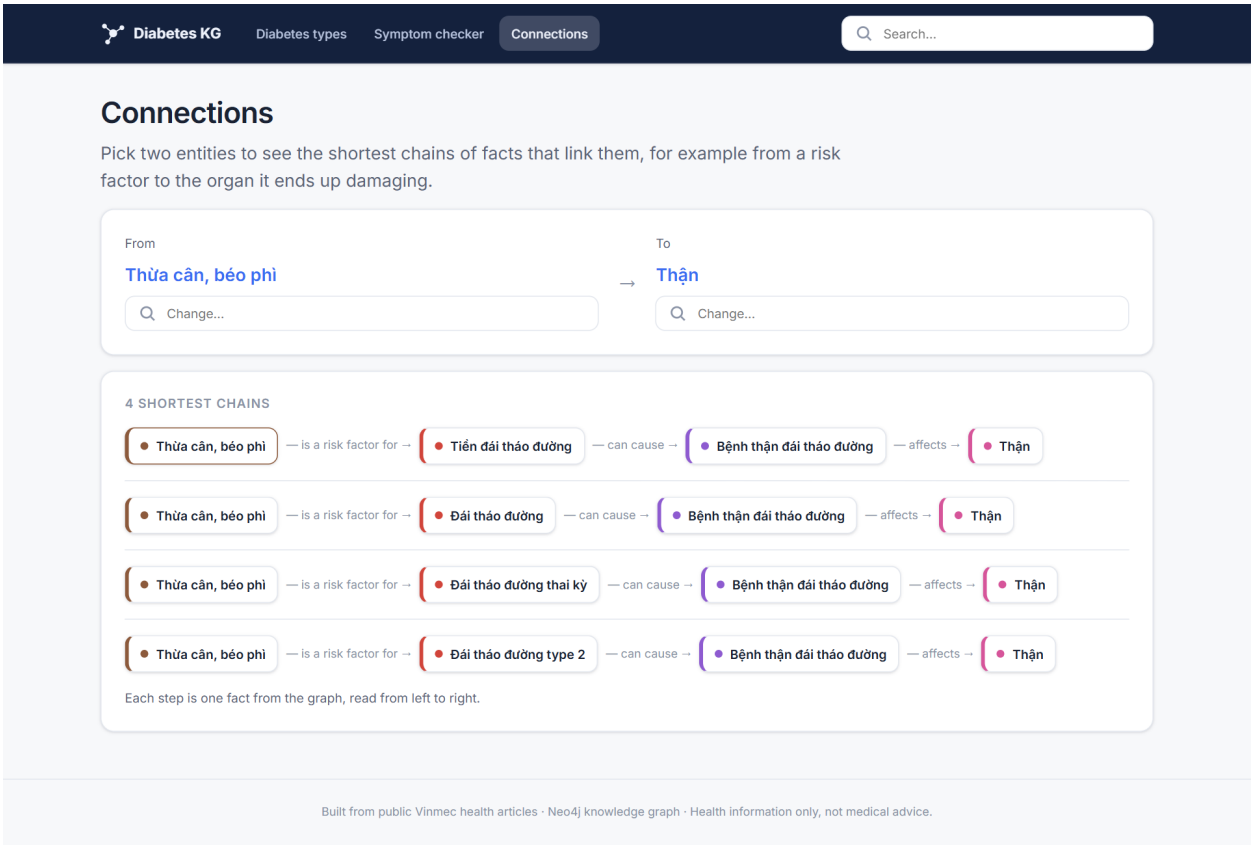



Figure 21: The *Connections* page: shortest chains of facts from obesity (*Thừa cân, béo phì*) to the kidney (*Thận*).

The *Connections* page (Figure 21) takes two entities and shows the shortest chains of facts between them. From obesity to the kidney there are four chains of three steps, one through each diabetes type that lists obesity as a risk factor, for example:

> *Thừa cân, béo phì* — is a risk factor for → *Đái tháo đường type 2* — can cause → *Bệnh thận đái tháo đường* — affects → *Thận*.

Each node of a chain is drawn as a card with a border in its type colour (brown for the risk factor, red for the diabetes type, purple for the complication, pink for the organ), so the kind of each step is visible without reading it. Each step is one fact, worded in the right direction: the stored fact is "type 2 HAS_RISK_FACTOR obesity", and since the path walks it backwards the site writes "is a risk factor for". A chain like this is the kind of explanation a graph offers and a document list does not: each step comes from different articles, joined because they share a node.

# 9 Error Analysis

The quality check of Section 7.2 found two wrong facts in fifty. This section explains why, what was changed, what the change cost, and which other errors are known.

## 9.1 The Two Errors in the Sample

Table 26: The two wrong facts in the sample and what the articles actually said.

| Wrong fact | What the supporting articles said |
|---|---|
| Đái tháo đường HAS_RISK_FACTOR Rối loạn mỡ máu (support 16) | High cholesterol is listed as a risk factor of the *kidney* and *eye* complications, under headings such as "Những ai dễ bị bệnh thận đái tháo đường?" and "Ai có nguy cơ mắc bệnh võng mạc đái tháo đường?", and as a cause of itching ("Một số nguyên nhân gây ngứa bàn tay và bàn chân") |
| Đái tháo đường HAS_SYMPTOM Đau bụng (support 18) | Abdominal pain is described as a symptom of *ketoacidosis* and of nerve damage in the gut, under headings such as "Biến chứng thần kinh" and "Kiểm soát biến chứng và phục hồi chức năng" |

Both errors share one cause: a fact belonging to a **complication** was attached to the disease itself. The rule allowed it because the headings held an allowed keyword (*nguy cơ*, *nguyên nhân*) or because, in the first version, complication sections could also yield symptoms. The rule looked at *what kind* of information a heading announces but not *whose* symptoms or risk factors they are.

Two observations sharpened the diagnosis. First, both wrong facts had high support (16 and 18), so the support filter could not catch them: many articles discuss diabetic complications and each contributed a wrong vote. Agreement removes random errors; it cannot remove one the rules make systematically. Second, both errors were in the two relation types where ownership matters: a complication always belongs to diabetes, but a symptom may belong to diabetes or to one of its complications.

## 9.2 The Fix

The fix is `about_disease()` of Section 4.3.4: for symptoms and risk factors the heading must name diabetes or be short and generic, must name no other topic (a complication, an organ, itching, stroke, coma…), and must not be about reducing a risk; and complication sections no longer yield symptoms. Rerunning extraction took the graph from 345 to 339 facts.

## 9.3 What the Fix Cost: Precision Versus Recall

Table 27: The six facts removed by the stricter rule.

| Removed fact | Was the removal right? |
|---|---|
| type 1 HAS_SYMPTOM Tê bì tay chân | yes: numbness is the nerve-damage complication, not a type 1 symptom |
| type 2 HAS_SYMPTOM Vã mồ hôi | yes: sweating is a symptom of low blood sugar |
| Tiền đái tháo đường HAS_RISK_FACTOR Tuổi cao | no: a true fact (sample #5) was lost |
| Tiền đái tháo đường HAS_RISK_FACTOR Tăng huyết áp | no: a true fact (sample #15) was lost |
| Đái tháo đường HAS_RISK_FACTOR Dùng corticoid kéo dài | no: a true fact was lost |
| type 2 HAS_RISK_FACTOR Hút thuốc lá | no: a true fact was lost |

Two of the six removals were genuine errors the sample had not happened to contain. The other four were true facts. Each had exactly one good supporting article and one weak one (the age and blood-pressure facts for prediabetes came partly from "Chăm sóc các vấn đề tim mạch ở người bệnh tiểu đường và tiền tiểu

đường", whose risk-factor bullets sit under a heading about heart risk). The stricter rule removed the weak vote, support fell to 1, and the two-article minimum removed the fact. One visible consequence in Table 6: the risk factor *Dùng corticoid kéo dài* lost its only fact and left the graph.

The two original errors stayed in the graph with far less support: 16 → 3 articles for high cholesterol, 18 → 6 for abdominal pain. The remaining articles do state them under genuine diabetes headings ("Nguyên nhân và yếu tố nguy cơ gây bệnh tiểu đường: …Rối loạn mỡ máu"; abdominal pain in symptom lists for diabetes in children, where it is the sign of ketoacidosis at onset). By the definition used here, that the articles state it, they are now properly supported. The weakness lay in the supporting articles, not in the facts.

This is the **precision/recall trade-off** every extraction system faces: strict rules give fewer wrong facts but miss more true ones, loose rules find more true facts and admit more wrong ones, and no setting suits every use. Hence support stays on every fact and is not used only once as a filter: a careful application may demand three or five articles, someone exploring may see everything, and the website exposes exactly that choice.

### 9.4 Other Known Errors

Fifty facts cannot expose every error. Inspecting the queries and the graph revealed four more.

**A wrong side effect.** The graph contains *Insulin HAS_SIDE_EFFECT Nhiễm toan ceton* (ketoacidosis), supported by five articles. It is wrong: ketoacidosis results from a *lack* of insulin and is *treated* with insulin. The filter of Section 4.3 skips sentences with *thiếu* or *không đủ*, but articles link insulin and ketoacidosis in other ways that the rule "drug + complication + *gây* in one sentence" cannot separate:

- insulin as the **treatment**: "nhiễm toan ceton …có thể *gây* nguy hiểm đến tính mạng nếu như không được xử trí kịp thời bằng insulin";
- another drug causing it **together with** insulin: "Danazol: dùng cùng với Insulin do có thể *gây* tăng glucose máu và nhiễm toan ceton";
- hormones that **oppose** insulin: "Những hormone này …có tác dụng ngược lại với insulin …sẽ *gây* ra toan ceton".

In each sentence *gây* belongs to something other than insulin. Deciding *what* causes *what* needs the sentence's grammar, which cue words cannot supply. Sentence-level relation extraction is the weakest part of the method, and Section 11 proposes a language model for it.

**An alias with a second meaning.** The fruit alias *táo* (apple) also matches *táo bón* (constipation). Word boundaries cannot help since *táo* is a whole word in both. The section rule limits harm (a food counts only in diet sections), but a diet section mentioning constipation would cast a false vote for low-sugar fruit.

**A concept hidden by a longer one.** The vocabulary has *Hạ đường huyết sơ sinh* (newborn hypoglycaemia) apart from *Hạ đường huyết*. Articles rarely use the full phrase; they write "trẻ sinh ra có thể bị hạ đường huyết", which matches the general entity. The newborn entity never reached two articles and is absent from the graph, while its facts are counted under general hypoglycaemia (which is why hypoglycaemia tops the gestational complications).

**Dominance of the general node.** Since the subject comes from the title alone, the general diabetes node collects most facts (Figure 7). No single fact is wrong, but type-specific questions are less complete than they could be.

## 10 Limitations

The lecture names four ways a Knowledge Graph can fail: missing entities, incorrect relationships, duplicate entities and outdated information. Each applies here in a specific way, and a few more limits come from

design choices.

**Missing entities (unknown recall).** The vocabulary is fixed at 106 concepts written 320 ways. Anything outside it can never be found, however often articles discuss it: a newer drug or insulin brand, a rarer complication, a traditional remedy, a specific test. The evaluation measured how many extracted facts are correct (precision) but not how many true facts were missed (recall). Measuring recall would mean annotating a set of articles by hand with every fact they contain, which was beyond this project's scope.

**Incorrect relationships.** About 4% of sampled facts were wrong (likely range 1–13%), mostly complication facts attached to the disease. Headings are a strong but imperfect signal of meaning, and cue-word side-effect extraction from single sentences is weaker still (Section 9).

**Duplicates and granularity.** Aliases merge near-synonyms on purpose (*tê bì*, *dị cảm*, *kiến bò* all become *Tê bì tay chân*) and sometimes merge what a doctor would separate, such as hypoglycaemia of mother and newborn. A concept spelled in a way the vocabulary does not know is missed and not duplicated, so true duplicates are rare.

**Outdated information.** All articles carry a 2024 date reflecting Vinmec's republication of its archive, not when each was written, and an old article counts the same as a new one. Medical advice changes (new drug classes, revised targets) and the graph has no notion of time. A real system would recrawl regularly and date each fact.

**Coarse subjects.** The form of diabetes comes only from the title, so many facts that really concern type 2 sit on the general node.

**Fixed food advice.** Whether a food is recommended or avoided is decided by the food, not read from the article, so conditional advice ("in moderate amounts", "choose low-sugar varieties") cannot be represented.

**One source.** Everything comes from one hospital's website. That keeps the graph consistent but inherits its emphases and gaps, and "correct" here means "stated by Vinmec", not "checked against current clinical guidelines".

**Hand-added facts.** Drug classes, affected organs and diagnostic thresholds were added by hand. They are labelled `curated` and are standard medical knowledge, but were not learned from the articles.

**Not medical advice.** The graph summarises public health articles. The symptom checker shows which forms of diabetes the articles associate with symptoms; it diagnoses no one, and the website says so where it matters.

# 11 Conclusion and Future Work

## 11.1 What Was Achieved

The project built a Knowledge Graph of diabetes knowledge from Vietnamese health articles, stored it in Neo4j and made it explorable through a website.

- **Data (R1).** 1,197 candidate pages were found through Vinmec's sitemap; a relevance filter on accented text removed accent collisions and off-topic pages, leaving 1,096 diabetes articles (1.13 million words) saved section by section.
- **Knowledge Graph (R2, R3).** An extractor combining a hand-written vocabulary, the articles' section headings and agreement between articles produced 127 entities of 10 types and 339 facts of 12 relation

types, stored as `subject, relation, object` CSV with support and sources, well above the lecture's minimums of 50 entities, 100 relationships, 5 relation types and 3 entity types.

- **Neo4j and queries (R4, R6).** The graph sits in Neo4j with evidence on every relationship and source articles as nodes. Fifteen Cypher queries, covering one-hop, two-hop, aggregation, path and provenance, answer the questions the system was designed for; `graph_facts` retrieves facts around an entity within one or two hops, and the evaluation showed why the bound is needed (5 facts at one hop, 230 at two).
- **Visualisation (R5).** The graph was shown whole, by neighbourhood, by drug family and by diabetes type, and analysed with degree, directed and undirected PageRank and article mentions.
- **Website (R7).** A small API and a React client let anyone search the graph, read an entity's facts with sources, compare diabetes types, check symptoms and find the chains of facts joining two entities.
- **Evaluation (R8, R9).** 48 of 50 randomly sampled facts were correct (0.96; likely range 0.87–0.99). The two errors had one cause, which was fixed, and the fix's cost in lost true facts was measured and explained as a precision/recall trade-off.

The main lesson is that **structure already in the text can do most of the work**. Health articles are organised under standard headings, and using them to decide what kind of fact a sentence may hold, together with requiring independent articles to agree, produced a reliable graph without any trained model. The audit also showed the limit of agreement: it removes random noise but not an error the rules make systematically, and only looking at the errors revealed that.

### 11.2 Future Work

1. **Measure recall.** Annotate a sample of articles with every fact they state and measure how many the extractor finds, which would quantify the project's main unknown.
2. **Grow the vocabulary semi-automatically.** Frequent noun phrases under symptom or treatment headings that match no entry are candidates for new entities, needing only a person's approval.
3. **Better relation extraction.** Use a language model to extract relations sentence by sentence ("does this sentence say X causes Y, and is X taken or lacking?"), checked against the section rule. This would address the side-effect errors and allow relations such as *food prevents complication*.
4. **Knowledge Graph + RAG.** Connect the graph to the retrieval-augmented chat system of Assignment 08: the graph supplies facts and their paths, article chunks supply detailed explanation, and the answer cites both.
5. **Freshness.** Recrawl on a schedule, store each article's date on its facts, and prefer recent evidence when articles disagree.

## A Reproduction

All commands run from the `graph_01/` folder. On Windows, set `PYTHONUTF8=1` first so Python reads and writes the Vietnamese text as UTF-8.

```
# environment
python -m venv .venv && source .venv/bin/activate        # Windows:
↪  .venv\Scripts\activate
python -m pip install -r requirements.txt

# 1. crawl (about 25 minutes, resumable)  -> dataset/raw/pages/*.json,
↪  dataset/raw/index.csv
python kg/crawl.py
```

```
# 2. extract                                    ->
↪  dataset/processed/{entities,triples,articles,mentions}.csv
python kg/extract.py

# 3. Neo4j (Docker) and load
docker run -d --name neo4j-diabetes -p 7474:7474 -p 7687:7687 \
    -e NEO4J_AUTH=neo4j/diabetes123 -v "$PWD/neo4j_data:/data" neo4j:5
python kg/load.py

# 4. retrieval, drawings, quality sample
python kg/retriever.py "Metformin" --hops 1
python kg/visualize.py
python kg/evaluate.py          # draws a new sample; overwrites
↪  evaluation/manual_audit.csv

# 5. notebook (run from notebook/ or the project root)
jupyter nbconvert --to notebook --execute --inplace
↪  notebook/01_diabetes_kg.ipynb

# 6. website
uvicorn api.main:app --reload --port 8000      # API, docs at
↪  http://localhost:8000/docs
cd web && npm install && npm run dev           # http://localhost:5173

# 6b. or everything at once with Docker (Neo4j + loader + API + website, no
↪  Python or Node needed)
docker compose up --build                      # website http://localhost:8080,
↪  API docs :8000/docs

# 7. report
python report/extract_figures.py && python report/make_tables.py
cd report && xelatex Graph_01.tex && xelatex Graph_01.tex
```

The Cypher queries of Section 7.1 are in `kg/queries.cypher` and can be pasted into Neo4j Browser at `http://localhost:7474` (user `neo4j`, password `diabetes123`).